

GreyNOC

Defensive Scripting Manual

Android Termux, Linux, Bash, Python, building, deployment, guided lessons, labs, quizzes, and team-based operations

Version v1.1.0 | Release date 2026-06-18 | Public defensive training edition

Responsible-use boundary: this manual is for authorized administration, defensive monitoring, secure operations, incident triage, authorized team exercises, and education in systems you own or have explicit written permission to operate.

Document Control

Document ID	GN-PUB-DSM-001
Title	GreyNOC Defensive Scripting Manual
Classification	Public - Defensive use only
Template	GN-TPL-PUB-001
Owner	GreyNOC Operations
Author	GreyNOC
Version	v1.1.0
Release date	2026-06-18
Supersedes	v1.0.0
Review cycle	Per release or every 6 months, whichever comes first
Distribution	Public; GreyNOC operators, defenders, administrators, authorized lab learners
Responsible-use boundary	Authorized administration, defensive monitoring, secure operations, incident triage, authorized team exercises, and education on owned or explicitly permitted systems. Excludes stealth, unauthorized access, credential theft, malware, and exploitation of third-party systems.

Version History

Version	Date	Summary of changes
v1.0.0	2026-06-18	Initial public release: foundations, Bash, Termux, Linux automation, Python, secure design, build/deploy, 99 drill cards, capstones, and quiz.
v1.1.0	2026-06-18	Added Part 9 (Red, Blue, Purple, White team-based scripting) and three team capstones. Fixed the final quiz (was one question repeated 60 times) to 30 distinct questions. Rebuilt the capstone table to remove overlapping text. Corrected the strict-mode IFS code (tab glyph rendered as a box). Added this Document Control and Version History.

Manual Map

This manual is an intensive operator course. It builds from foundations and safe operator mindset, through Bash, Termux, Linux service automation, Python, secure design, and build/deploy, into compact drill cards, team-based operations scripting, and finally capstone labs and quizzes.

- Part 1: foundations and safe operator mindset
- Part 2: Bash defensive scripting
- Part 3: Android Termux scripting and field operations
- Part 4: Linux automation, systemd, cron, services, inventory, logs
- Part 5: Python helper tools and packaging
- Part 6: secure script design
- Part 7: building, packaging, deploying, and releasing
- Part 8: operator micro-lessons and drill cards
- Part 9: team-based operations scripting (Red, Blue, Purple, White)

How to study this manual

- 1 Read the purpose and safety note first.
- 2 Type examples by hand in a disposable lab when learning.
- 3 Run static checks and dry-run mode before changing state.
- 4 Answer the quiz before looking back at the lesson.
- 5 Maintain a lab notebook with command, result, issue, and fix.

Source Analysis - Operator Guides Repository

The Operator Guides repository identifies itself as a public documentation repository for defensive cybersecurity, system administration, and operator-focused field references. The repository goal is practical, responsible-use guidance for configuring systems, hardening environments, monitoring networks, triaging security events, and documenting repeatable workflows. The repository safety standard requires use only on systems, devices, accounts, networks, and lab environments that the operator owns or has explicit written permission to administer, monitor, assess, or test.

Repo signal	How this manual uses it
Defensive field references	Procedures are limited to authorized hardening, monitoring, administration, and training.
Linux-on-Android guide present	Termux and Android sections are first-class field operator workflows.
Cyber hygiene and privacy guides present	Includes privacy, data minimization, logging, and safe output handling.
Suggested guide format	Each lesson uses purpose, procedure, verification, troubleshooting, and quiz elements.
Team exercise references	Part 9 keeps red, blue, purple, and white work inside authorized scope and deconfliction.

Safety boundary: this manual intentionally excludes stealth, unauthorized access, credential theft, malware, exploitation against third-party systems, bypassing access controls, and instructions that would help hide activity.

Part 1 - Foundations

A defensive script is a small operational product. It has users, failure modes, maintenance needs, and security impact. The foundations in this part give GreyNOC operators a shared standard for writing scripts that are safe, repeatable, documented, and fit for authorized environments.

Chapter operator checklist

- Can the reader explain the defensive purpose?
- Can the reader run the lab safely?
- Can the reader verify output?
- Can the reader roll back?
- Can the reader explain what data is handled?

Foundation Lesson: Safety, Authorization, and Scope

Purpose. Every script in this manual is written for defensive operations, authorized administration, home-lab learning, and repeatable system-care work. A defensive script should reduce toil, increase visibility, preserve evidence, improve consistency, or help an operator make a safer decision. It should not conceal itself, bypass user consent, defeat platform controls, harvest private data, or touch systems outside written permission. Key points: State owner, asset, objective, and rollback before running any script.; Prefer read-only discovery before making changes.; Log actions, timestamps, inputs, outputs, and errors.; Never store secrets in source code, screenshots, chat logs, or shell history.; Test in a lab account or disposable VM before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
purpose="defensive operator standard"
asset_scope="owned-or-authorized systems only"
echo "purpose=$purpose"
echo "scope=$asset_scope"
echo "verify, log, document, rollback"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.

- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the authorized scope? | What is the rollback? | What evidence proves success? | What data is sensitive?

Foundation Lesson: Operator Script Lifecycle

Purpose. Defensive scripting is not just writing commands. It is a lifecycle: define the outcome, select the safest interface, build a small working version, add validation, add logging, add rollback, test failure cases, deploy with change control, observe the result, and document lessons learned. Key points: Plan: problem, scope, platforms, permissions, data handled.; Prototype: smallest safe command that proves the idea.; Harden: input validation, strict mode, traps, timeouts, idempotence.; Package: README, usage, dependencies, checksums, version.; Deploy: staged rollout, monitoring, rollback, owner sign-off.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
purpose="defensive operator standard"
asset_scope="owned-or-authorized systems only"
echo "purpose=$purpose"
echo "scope=$asset_scope"
echo "verify, log, document, rollback"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.

- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the authorized scope? | What is the rollback? | What evidence proves success? | What data is sensitive?

Foundation Lesson: Threat Modeling for Scripts

Purpose. A script can become a risk if it runs too broadly, trusts unvalidated input, leaks logs, changes permissions, or fails open. Treat scripts as operator tools with their own attack surface. Ask what the script can read, write, delete, disclose, schedule, or execute, and design limits around those powers. Key points: What credentials does it need?; What files can it modify?; What data does it print to logs?; What happens on partial failure?; Can it be re-run safely?; Who reviews updates?

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
purpose="defensive operator standard"
asset_scope="owned-or-authorized systems only"
echo "purpose=$purpose"
echo "scope=$asset_scope"
echo "verify, log, document, rollback"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.

- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the authorized scope? | What is the rollback? | What evidence proves success? | What data is sensitive?

Foundation Lesson: Defensive Scripting Principles

Purpose. Good operator scripts are boring, predictable, reversible, and documented. They should prefer built-in tools, standard formats, explicit arguments, clear errors, and no hidden side effects. A great script can be handed to another operator who can understand its purpose without calling the author. Key points: Make dangerous actions opt-in with named flags.; Use dry-run mode for write actions.; Exit non-zero on failure.; Use structured output for machines and readable summaries for humans.; Keep platform-specific logic isolated.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
purpose="defensive operator standard"
asset_scope="owned-or-authorized systems only"
echo "purpose=$purpose"
echo "scope=$asset_scope"
echo "verify, log, document, rollback"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.

- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the authorized scope? | What is the rollback? | What evidence proves success? | What data is sensitive?

Foundation Lesson: GreyNOC Documentation Standard

Purpose. GreyNOC operator material should be practical, professional, responsible, and easy to verify. Each guide or script should list purpose, audience, prerequisites, safety notes, procedure, verification, troubleshooting, and references. Key points: Use a short purpose statement.; Include supported platforms and tested versions.; Provide validation commands.; Include rollback where applicable.; Version every release and keep a QA record.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
purpose="defensive operator standard"
asset_scope="owned-or-authorized systems only"
echo "purpose=$purpose"
echo "scope=$asset_scope"
echo "verify, log, document, rollback"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the authorized scope? | What is the rollback? | What evidence proves success? | What data is sensitive?

Part 2 - Bash for Defensive Operators

Bash is the fastest path from manual command to repeatable operator workflow on Linux and Termux. Use Bash for orchestration, file handling, service checks, log collection, and small wrappers. Move to Python when parsing or testing becomes too complex.

Chapter operator checklist

- Can the reader explain the defensive purpose?
- Can the reader run the lab safely?
- Can the reader verify output?
- Can the reader roll back?
- Can the reader explain what data is handled?

Bash Lesson: Bash Strict Mode and Defensive Defaults

Purpose. Use strict mode when a shell script controls files, services, or deployment. Strict mode turns common mistakes into visible failures instead of quiet damage.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
IFS=$'\n\t'
trap 'echo "ERROR line=$LINENO command=$BASH_COMMAND" >&2' ERR
main() {
    echo "starting defensive task"
}
main "$@"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why is `set -u` useful? | What does `pipefail` change? | When might strict mode need a local exception?

Bash Lesson: Argument Parsing with getopt

Purpose. Use explicit flags instead of positional mystery. A script should reject unknown flags and print help when required values are missing.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
usage(){ echo "Usage: $0 -t TARGET -o OUTDIR [-n]"; }
dry_run=false
while getopt ":t:o:nh" opt; do
  case "$opt" in
    t) target="$OPTARG" ;;
    o) outdir="$OPTARG" ;;
    n) dry_run=true ;;
    h) usage; exit 0 ;;
    :) echo "Missing value for -$OPTARG" >&2; usage; exit 2 ;;
    \?) echo "Unknown flag -$OPTARG" >&2; usage; exit 2 ;;
  esac
done
: "${target:?target required}"
: "${outdir:?outdir required}"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.

- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why are named flags safer? | What exit code should usage errors use? | How would you add a --json option?

Bash Lesson: Input Validation

Purpose. Validate before action. For paths, resolve directories. For hostnames, allow only expected characters. For IP addresses, use trusted tools where possible.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
safe_name_re='^[A-Za-z0-9._-]+$'
if [[ ! "$asset_name" =~ $safe_name_re ]]; then
    echo "Invalid asset name" >&2
    exit 2
fi
if [[ ! -d "$outdir" ]]; then
    mkdir -p -- "$outdir"
fi
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What characters are intentionally excluded? | Why quote variables? | Why use `--` before path arguments?

Bash Lesson: Logging with Rotation

Purpose. Operator scripts should create useful evidence without flooding disk. Log both the high-level event and the exact command result when appropriate.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
log_dir="${HOME}/.local/state/greynoc/logs"
mkdir -p "$log_dir"
log_file="$log_dir/defensive-task-$(date +%Y%m%d).log"
log(){ printf '%s %s'
' "$(date -Is)" "$*" | tee -a "$log_file"; }
find "$log_dir" -type f -name '*.log' -mtime +14 -delete
log "task=start user=$(id -un) host=$(hostname)"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.

- Skipping peer review because the script is short.

Knowledge check What should not be logged? | Why use ISO timestamps? | How long should logs be retained?

Bash Lesson: File Backups Before Change

Purpose. Before editing configuration, copy the original with timestamp and permissions intact. A rollback file should be obvious and nearby.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
backup_file(){
  local src="$1"
  [[ -f "$src" ]] || { echo "Missing file: $src" >&2; return 1; }
  local stamp; stamp=$(date +%Y%m%d-%H%M%S)
  cp -a -- "$src" "${src}.bak.${stamp}"
  echo "Backup: ${src}.bak.${stamp}"
}
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why use `cp -a`? | Where should backups live? | How would you verify the backup?

Bash Lesson: Atomic Writes

Purpose. Write to a temporary file and then move it into place. This prevents partial files if the script is interrupted.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
tmp=$(mktemp)
printf '%s'
' "new content" > "$tmp"
chmod 0644 "$tmp"
mv -- "$tmp" "$target_file"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why is `mv` usually safer than direct overwrite? | What directory should temp files be created in? | How do permissions affect atomic replacement?

Bash Lesson: Traps and Cleanup

Purpose. Temporary files, lock files, and mounted resources should be cleaned up even when a script fails.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
tmpdir=$(mktemp -d)
cleanup(){ rm -rf -- "$tmpdir"; }
trap cleanup EXIT INT TERM
echo "working in $tmpdir"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Which signals are handled? | What should never be removed by cleanup? | How do you test cleanup?

Bash Lesson: Lock Files and Concurrency

Purpose. Use locks when a script touches shared state. Two backup jobs should not rotate the same files at the same time.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.

- 6 Verify final state with an independent command and document the result.

Reference implementation

```
lock="/tmp/greynoc-backup.lock"
exec 9>"$lock"
if ! flock -n 9; then
    echo "Another run is active" >&2
    exit 75
fi
# protected work here
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What problem does `flock` solve? | What should the script do if lock acquisition fails? | Why choose exit code 75?

Bash Lesson: Service Health Check

Purpose. A health script should distinguish process running, port listening, and application responding. Each is a different signal.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
svc="ssh"
if systemctl is-active --quiet "$svc"; then
    echo "$svc active"
else
    echo "$svc inactive" >&2
    systemctl status "$svc" --no-pager | tail -40
    exit 1
fi
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why is process status not enough? | How would you add a TCP check? | How would you add an HTTP check?

Bash Lesson: Disk Guardrail

Purpose. Disk-full conditions create outages and lost logs. A defensive disk guard should alert early and print the largest safe candidates for cleanup.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
threshold=85
while read -r use mount; do
  pct=${use%%}%
  if (( pct >= threshold )); then
    echo "WARN disk usage ${use} at ${mount}"
    find "$mount" -xdev -type f -size +100M -printf '%s %p'
  ' 2>/dev/null | sort -nr | head
  fi
done < <(df -P | awk 'NR>1 {print $5, $6}')
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why avoid automatic deletion by default? | What is `-xdev` for? | What output would you send to a ticket?

Bash Lesson: Log Collection Bundle

Purpose. During incident triage, collect logs consistently and preserve timestamps. Avoid collecting secrets or excessive personal data.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
case_id="${1:?case id required}"
out="greynoc-case-${case_id}-${date +%Y%m%d-%H%M%S}"
mkdir -p "$out"
journalctl -S -24h --no-pager > "$out/journal-last24h.txt"
cp -a /var/log/auth.log* "$out/" 2>/dev/null || true
tar --sort=name --mtime='UTC 2020-01-01' -czf "${out}.tar.gz" "$out"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why include case id? | What should be excluded? | How do you preserve chain of custody?

Bash Lesson: Network Snapshot

Purpose. A network snapshot records local listening services and active connections for defenders. It is not a scan of third-party networks.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
out="net-snapshot-$(hostname)-$(date +%Y%m%d-%H%M%S).txt"
{
  date -Is
  ip addr
  ip route
  ss -tulpen
  ss -tanp
} > "$out"
chmod 0600 "$out"
echo "Wrote $out"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why chmod 0600? | What data may be sensitive? | How often should this run?

Bash Lesson: User and Permission Review

Purpose. User review scripts should list accounts, groups, shells, sudoers entries, and recent login activity without changing anything.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
{
  echo "== users =="; getent passwd
  echo "== groups =="; getent group
  echo "== sudoers snippets =="; ls -la /etc/sudoers.d 2>/dev/null || true
  echo "== recent logins =="; last -n 20
} > user-review.txt
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What indicates risk? | Why avoid parsing /etc/passwd manually when getent exists? | What needs human review?

Bash Lesson: Patch Readiness Check

Purpose. Before patching, capture OS version, package manager health, disk capacity, service owners, and rollback notes.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
{
  echo "Host: $(hostname)"
  cat /etc/os-release 2>/dev/null
  df -h
  systemctl --failed --no-pager || true
  apt-get -s upgrade 2>/dev/null || dnf check-update || true
} > patch-readiness.txt
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What does simulation mode provide? | What makes a patch window safe? | When do you stop?

Bash Lesson: Safe Remote Execution Wrapper

Purpose. Remote admin scripts should be transparent, logged, and limited to named hosts under authorization. Avoid spraying commands across unknown ranges.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation


```
hosts_file="${1:?hosts file required}"
cmd="uptime"
while IFS= read -r host; do
    [[ -z "$host" || "$host" == \#* ]] && continue
    echo "== $host =="
    ssh -o BatchMode=yes -o ConnectTimeout=5 "$host" "$cmd"
done < "$hosts_file"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why require a hosts file? | What does BatchMode prevent? | How do you record authorization?

Bash Lesson: Configuration Drift Check

Purpose. A drift script compares actual state with expected state and reports differences without automatically forcing convergence unless approved.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
expected_sha="$(sha256sum baseline.conf | awk '{print $1}')"
actual_sha="$(sha256sum /etc/example.conf | awk '{print $1}')"
if [[ "$actual_sha" != "$expected_sha" ]]; then
    echo "DRIFT /etc/example.conf expected=$expected_sha actual=$actual_sha"
    diff -u baseline.conf /etc/example.conf || true
fi
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why hash first? | Why report before remediation? | How do you approve changes?

Bash Lesson: ShellCheck and Static Review

Purpose. ShellCheck finds common shell bugs such as unsafe quoting, unreachable branches, and portability mistakes. Treat warnings as teaching prompts, not insults.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
shellcheck scripts/*.sh
shfmt -w scripts/*.sh
bash -n scripts/*.sh
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What does bash -n miss? | Why use a formatter? | When is a ShellCheck disable acceptable?

Part 3 - Android Termux Defensive Scripting

Termux is useful for owned Android devices, mobile field notes, lightweight automation, SSH administration of the phone itself, and Linux-on-Android learning. Termux scripts must respect Android app sandboxing, battery management, storage permissions, notification visibility, and the fact that the phone is not a traditional always-on server.

Chapter operator checklist

- Can the reader explain the defensive purpose?
- Can the reader run the lab safely?
- Can the reader verify output?
- Can the reader roll back?
- Can the reader explain what data is handled?

Termux Lesson: Termux Defensive Operating Model

Purpose. Termux gives an Android device a user-space terminal environment. Use it for learning, local automation, personal device checks, lightweight monitoring, and field notes. Android is not a full traditional Linux distribution, so scripts must respect app sandboxing, battery rules, storage permissions, and package differences.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
pkg update && pkg upgrade
pkg install bash coreutils findutils grep gawk sed tar openssh termux-api
termux-setup-storage
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.

- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What can Termux do without root? | What Android limits remain? | Why avoid pretending Termux is a server?

Termux Lesson: Termux Keepalive Script

Purpose. A keepalive script can help an authorized personal lab session stay observable while you work. It should be visible, logged, and easy to stop. Do not use keepalive behavior to hide activity, bypass ownership, or defeat policy.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/data/data/com.termux/files/usr/bin/bash
set -Eeuo pipefail
log="$HOME/.local/state/greynoc/termux-keepalive.log"
mkdir -p "$(dirname "$log")"
termux-wake-lock || true
trap 'termux-wake-unlock || true; echo "$(date -Is) stop" >> "$log"' EXIT
while true; do
  echo "$(date -Is) alive battery=$(termux-battery-status 2>/dev/null | tr "
  " " ")" >> "$log"
  sleep 300
done
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.

- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check How do you stop it? | What is logged? | Why should this be visible to the owner?

Termux Lesson: Termux Boot Startup

Purpose. Termux:Boot can run scripts after device boot when installed from the same app source as Termux. Boot scripts should be short, safe, and should start user-approved tasks only.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
mkdir -p ~/.termux/boot
cat > ~/.termux/boot/greynoc-start.sh <<'EOF'
#!/data/data/com.termux/files/usr/bin/bash
mkdir -p "$HOME/.local/state/greynoc"
echo "$(date -Is) boot observed" >> "$HOME/.local/state/greynoc/boot.log"
EOF
chmod +x ~/.termux/boot/greynoc-start.sh
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What should not be started at boot? | Where are boot logs kept? | How do you disable it?

Termux Lesson: Termux Service Supervisor Pattern

Purpose. Long-running Termux jobs need a supervisor pattern so operators can start, stop, and inspect them. Keep scripts in one folder and write PID files.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
svc_dir="$HOME/.local/share/greynoc/services"
mkdir -p "$svc_dir"
cat > "$svc_dir/start-heartbeat.sh" <<'EOF'
#!/data/data/com.termux/files/usr/bin/bash
nohup bash "$HOME/bin/heartbeat.sh" >> "$HOME/.local/state/greynoc/heartbeat.out"
  2>&1 &
echo $! > "$HOME/.local/state/greynoc/heartbeat.pid"
EOF
chmod +x "$svc_dir/start-heartbeat.sh"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is a PID file? | How do you verify the process? | Why separate service scripts from logs?

Termux Lesson: Termux Battery and Thermal Awareness

Purpose. Mobile scripts must be kind to battery and temperature. Prefer longer intervals, event-based checks, and clear stop controls.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
battery_json=$(termux-battery-status 2>/dev/null || echo '{}')
percent=$(printf '%s' "$battery_json" | jq -r '.percentage // 0' 2>/dev/null || echo 0)
if [ "$percent" -lt 25 ]; then
    echo "Battery low; skipping noncritical job"
    exit 0
fi
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why skip noncritical jobs? | What interval is reasonable? | How can charging state change behavior?

Termux Lesson: Termux Storage Backup

Purpose. Use Termux storage access to back up selected user-approved folders. Never sweep all storage blindly.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
src="$HOME/storage/shared/Documents/FieldNotes"
dst="$HOME/storage/shared/GreyNOC-Backups"
mkdir -p "$dst"
tar -czf "$dst/fieldnotes-$(date +%Y%m%d).tar.gz" -C "$src" .
sha256sum "$dst"/*.tar.gz > "$dst/SHA256SUMS.txt"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why select a narrow source folder? | Why hash backups? | Where should encrypted backups fit?

Termux Lesson: Termux SSH Admin Access

Purpose. SSH into Termux can be useful for personal lab administration. Use key auth, non-default ports when needed, and local network restrictions.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.

- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
pkg install openssh
mkdir -p ~/.ssh
chmod 700 ~/.ssh
# append your public key to ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
sshd -p 8022
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why avoid password auth? | What network is allowed? | How do you stop sshd?

Termux Lesson: Termux Android API Checks

Purpose. Termux:API can query battery, Wi-Fi, sensors, clipboard, and notifications. Use only data needed for the defensive task.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
termux-battery-status  
termux-wifi-connectioninfo  
termux-notification --title "GreyNOC" --content "Defensive check completed"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Which API outputs are sensitive? | How do notifications improve transparency? | What permissions are required?

Termux Lesson: Termux Package Baseline

Purpose. A package baseline helps rebuild your field device. Capture installed packages and key dotfiles.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
mkdir -p ~/greynoc-baseline  
pkg list-installed > ~/greynoc-baseline/pkg-list.txt  
cp ~/.bashrc ~/.profile ~/greynoc-baseline/ 2>/dev/null || true  
tar -czf ~/storage/shared/termux-baseline-$(date +%Y%m%d).tar.gz ~/greynoc-baseline
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.

- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What should not be included? | How do you restore? | Why keep a human-readable list?

Termux Lesson: Termux Network Self-Check

Purpose. Self-check scripts should inspect the Android device and the local authorized network only. Avoid scanning public or third-party systems.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
{
  date -Is
  ip addr 2>/dev/null || ifconfig
  termux-wifi-connectioninfo 2>/dev/null || true
  ss -tuln 2>/dev/null || true
} > "$HOME/storage/shared/greynoc-termux-netcheck.txt"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why is scope important? | What data could identify a private network? | How do you redact before sharing?

Termux Lesson: Termux Log Tail Notebook

Purpose. Use a small script to tail your own lab logs and create field notes. Separate observation from conclusions.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
log="$HOME/storage/shared/GreyNOC-FieldNotes/notes-$(date +%Y%m%d).md"
mkdir -p "$(dirname "$log")"
printf '
## %s
Observation:
Evidence:
Action:
' "$(date -Is)" >> "$log"
${EDITOR:-nano} "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.

- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why separate evidence from action? | What belongs in a field note? | How do you avoid private data leakage?

Termux Lesson: Termux Sync by Rsync

Purpose. Rsync can synchronize selected lab files to an authorized host. Always use key auth and dry-run first.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
rsync -avhn --delete ~/storage/shared/GreyNOC-FieldNotes/  
user@host:/srv/greynoc/fieldnotes/  
# after review:  
# rsync -avh --delete ~/storage/shared/GreyNOC-FieldNotes/  
user@host:/srv/greynoc/fieldnotes/
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What does -n do? | Why is --delete risky? | How do you restrict the remote account?

Termux Lesson: Termux Widget Launchers

Purpose. Termux:Widget can expose approved scripts as home-screen actions. Widget scripts should be idempotent and should show a notification when done.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
mkdir -p ~/.shortcuts
cat > ~/.shortcuts/GreyNOC_Status.sh <<'EOF'
#!/data/data/com.termux/files/usr/bin/bash
termux-notification --title "GreyNOC Status" --content "Running local status check"
bash "$HOME/bin/local-status.sh"
EOF
chmod +x ~/.shortcuts/GreyNOC_Status.sh
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why should widget scripts be safe to rerun? | What visible feedback is needed? | Who can tap the widget?

Termux Lesson: Termux Python Virtual Environments

Purpose. Use virtual environments to isolate Python tools. Pin dependencies for repeatable builds.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
pkg install python
python -m venv ~/.venvs/greynoc
. ~/.venvs/greynoc/bin/activate
python -m pip install --upgrade pip
python -m pip freeze > requirements.txt
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why isolate dependencies? | What belongs in requirements.txt? | How do you recreate the environment?

Termux Lesson: Termux Limits and Troubleshooting

Purpose. Common issues include storage permission, executable path differences, background process limits, package mirror trouble, and missing Linux filesystem paths. Troubleshoot by checking environment, paths, permissions, and logs.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
echo "$PREFIX"  
echo "$HOME"  
command -v bash python sshd  
ls -ld ~/storage/shared 2>/dev/null || termux-setup-storage  
pkg update
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is PREFIX? | Why is /usr not normal in Termux? | What should you document when a fix works?

Part 4 - Linux Automation and Operations

Linux defensive scripts often run under cron, systemd timers, SSH, CI runners, or service accounts. The operator must understand how noninteractive execution changes PATH, environment, permissions, logging, and error handling.

Chapter operator checklist

- Can the reader explain the defensive purpose?
- Can the reader run the lab safely?
- Can the reader verify output?
- Can the reader roll back?
- Can the reader explain what data is handled?

Linux Lesson: Systemd Service Unit for a Defensive Script

Purpose. A service unit makes a script manageable with start, stop, status, logs, and restart policy. Keep the script non-root when possible.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
[Unit]
Description=GreyNOC defensive collector
After=network-online.target
[Service]
Type=simple
User=greynoc
Group=greynoc
ExecStart=/opt/greynoc/bin/collector.sh
Restart=on-failure
RestartSec=30
NoNewPrivileges=true
PrivateTmp=true
[Install]
WantedBy=multi-user.target
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.

- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why use a dedicated user? | What does NoNewPrivileges do? | Where do logs go?

Linux Lesson: Systemd Timer Instead of Cron

Purpose. Timers provide calendar scheduling, monotonic scheduling, logs, dependency management, and missed-run handling. They are often better for managed Linux services.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
[Unit]
Description=Run GreyNOC daily health check
[Timer]
OnCalendar=*-*-* 03:15:00
Persistent=true
Unit=greynoc-health.service
[Install]
WantedBy=timers.target
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What does Persistent=true do? | How do you list timers? | When is cron simpler?

Linux Lesson: Cron with Logging

Purpose. Cron is simple and widely available. Always set PATH, redirect output, and avoid assumptions about interactive shell state.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
15 * * * * /opt/greynoc/bin/hourly-check.sh >> /var/log/greynoc/hourly-check.log 2>&1
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why set PATH? | Where does stderr go? | How do you disable safely?

Linux Lesson: Linux Hardening Check Script

Purpose. A hardening check reports state and recommends action. It should not silently change security posture without approval.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
checks=(  
  "systemctl is-enabled ufw"  
  "ss -tulpen"  
  "find /etc -xdev -perm -0002 -type f -ls"  
)  
for c in "${checks[@]}; do  
  echo "== $c =="  
  bash -c "$c" || true  
done
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Which checks need root? | What is safe to automate? | What needs policy approval?

Linux Lesson: Auditd Collection Helper

Purpose. Audit logs can be large and sensitive. Collect only the approved time range and case scope.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
since="${1:-today}"
out="audit-$(date +%Y%m%d-%H%M%S).txt"
ausearch -ts "$since" > "$out"
chmod 0600 "$out"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What sensitive data may appear? | How do you scope by time? | Who can read the output?

Linux Lesson: Journalctl Triage

Purpose. Journal queries are powerful for service triage. Always include time windows.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.

- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
svc="${1:?service required}"  
journalctl -u "$svc" -S -2h --no-pager --output=short-iso | tail -200
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why time-box logs? | What does -u select? | How do you export evidence?

Linux Lesson: Backup Verification

Purpose. Backups are only real when restore has been tested. A script should verify archive integrity and spot-check contents.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
archive="${1:?archive required}"
tar -tzf "$archive" >/dev/null
sha256sum "$archive" >> backup-ledger.txt
tar -tzf "$archive" | head -20
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why list contents? | What is a backup ledger? | How often should restore tests run?

Linux Lesson: Immutable Release Folder

Purpose. Deploy scripts into a versioned release folder and point a symlink at the active version. Rollback becomes a symlink change.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
release="/opt/greynoc/releases/1.0.0"
mkdir -p "$release"
cp -a bin lib README.md "$release/"
ln -sfn "$release" /opt/greynoc/current
/opt/greynoc/current/bin/healthcheck.sh
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why version folders? | What is the rollback command? | What should be immutable?

Linux Lesson: Least Privilege with Sudoers

Purpose. If a script needs one privileged command, grant that one command rather than full sudo. Test with a non-admin account.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
# /etc/sudoers.d/greynoc-health
greynoc ALL=(root) NOPASSWD: /usr/bin/systemctl status ssh, /usr/bin/journalctl -u
ssh *
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.

- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why is wildcard risky? | How do you validate sudoers syntax? | What alternative avoids sudo?

Linux Lesson: Secure Temp Files

Purpose. Use mktemp. Never build temp paths from predictable names in shared directories.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
tmpfile=$(mktemp)
trap 'rm -f -- "$tmpfile"' EXIT
printf '%s\n' "safe temporary content" > "$tmpfile"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is a race condition? | Why quote tmpfile? | What belongs in /run?

Linux Lesson: JSON Output for Automation

Purpose. Machines prefer structured output. Use jq to produce compact JSON that downstream tools can parse.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
jq -n --arg host "${hostname}" --arg time "$(date -Is)" --arg status "ok"
  '{host:$host,time:$time,status:$status}'
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why avoid parsing prose? | What fields belong in every event? | How do you validate JSON?

Linux Lesson: CSV for Human Operations

Purpose. CSV is useful for spreadsheet review but must be escaped correctly. Use Python when shell escaping gets complex.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.

- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
python3 - <<'PY'
import csv, socket, datetime
with open('inventory.csv','w',newline='') as f:
    w=csv.writer(f)
    w.writerow(['host','time','status'])
    w.writerow([socket.gethostname(), datetime.datetime.now().isoformat(), 'ok'])
PY
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why use a CSV library? | What encoding should be used? | How do you import safely?

Linux Lesson: Email or Notification Wrapper

Purpose. Notifications should be short, actionable, and not contain secrets. Put details in protected logs.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.

- 6 Verify final state with an independent command and document the result.

Reference implementation

```
subject="GreyNOC alert: disk threshold"
body="Host $(hostname) crossed disk threshold. See protected log path."
printf '%s\n' "$body" | mail -s "$subject" soc@example.invalid
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What should not be in an email? | Who receives alerts? | How do you deduplicate?

Linux Lesson: Endpoint Inventory Snapshot

Purpose. Inventory scripts should collect stable identifiers, package versions, kernel, storage, services, and security tooling state.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
{
  hostnamectl 2>/dev/null || hostname
  uname -a
  ip -brief addr
  systemctl list-unit-files --state=enabled --no-pager
  dpkg -l 2>/dev/null || rpm -qa
} > inventory.txt
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Which fields identify a device? | What data may be private? | How do you compare snapshots?

Linux Lesson: Container-Aware Scripts

Purpose. Scripts should detect containers and avoid host-only assumptions.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
if grep -qaE 'docker|kubepods|containerd' /proc/1/cgroup 2>/dev/null; then
  echo "container-like environment detected"
fi
if [ -f /.dockerenv ]; then echo "docker marker present"; fi
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why detect containers? | What commands fail in containers? | How does logging differ?

Linux Lesson: Linux File Integrity Baseline

Purpose. Hash critical config files and compare later. Store baselines in protected storage.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
find /etc/ssh /etc/sudoers.d -type f -print0 | sort -z | xargs -0 sha256sum >
baseline.sha256
sha256sum -c baseline.sha256
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.

- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What directories belong in scope? | Where should baselines be stored? | How do you handle expected changes?

Part 5 - Python for Defensive Automation

Python is the right tool when the job requires reliable parsing, APIs, structured data, tests, packaging, or cross-platform logic. Python scripts should still follow the same operator standard: least privilege, explicit scope, logs, validation, tests, packaging, and rollback.

Chapter operator checklist

- Can the reader explain the defensive purpose?
- Can the reader run the lab safely?
- Can the reader verify output?
- Can the reader roll back?
- Can the reader explain what data is handled?

Python Lesson: Python Script Skeleton

Purpose. Python is a strong choice when data parsing, APIs, CSV, JSON, or testing become awkward in shell. Start with argparse, logging, and a main function.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env python3
import argparse, logging, sys
def parse_args():
    p = argparse.ArgumentParser(description="GreyNOC defensive helper")
    p.add_argument("--dry-run", action="store_true")
    return p.parse_args()
def main():
    args = parse_args()
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
    logging.info("starting dry_run=%s", args.dry_run)
    return 0
if __name__ == "__main__":
    raise SystemExit(main())
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.

- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why return exit codes? | Why avoid top-level work? | How do you add tests?

Python Lesson: Python Subprocess Safely

Purpose. Avoid `shell=True` unless absolutely necessary. Pass arguments as a list so input is not interpreted by the shell.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
import subprocess
result = subprocess.run(["systemctl", "is-active", "ssh"], text=True,
                        capture_output=True)
print(result.returncode, result.stdout.strip(), result.stderr.strip())
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.

- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why avoid shell=True? | What should be logged? | How do timeouts help?

Python Lesson: Python Logging and Rotation

Purpose. Use rotating logs for long-running tools and include enough context to troubleshoot.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
import logging
from logging.handlers import RotatingFileHandler
handler = RotatingFileHandler("greynoc.log", maxBytes=1_000_000, backupCount=5)
logging.basicConfig(handlers=[handler], level=logging.INFO, format="%(asctime)s
    %(levelname)s %(message)s")
logging.info("collector started")
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What log size is safe? | What level is appropriate? | How do you avoid secrets?

Python Lesson: Python Config Files

Purpose. Keep configuration separate from code. Validate required keys before action.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
import json, pathlib
config = json.loads(pathlib.Path("config.json").read_text())
required = ["output_dir", "retention_days"]
missing = [k for k in required if k not in config]
if missing:
    raise SystemExit(f"Missing config keys: {missing}")
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why separate config? | What default values are safe? | How do you validate types?

Python Lesson: Python CSV Parsing

Purpose. Use the csv module. Never split CSV with string methods in production scripts.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
import csv
with open("assets.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["hostname"], row.get("owner", "unknown"))
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why DictReader? | How do you handle missing columns? | What encoding should be used?

Python Lesson: Python JSON Lines Events

Purpose. JSON Lines is useful for append-only event logs and streaming pipelines.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.

- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
import json, time, socket
event = {"ts": time.time(), "host": socket.gethostname(), "event": "healthcheck",
        "status": "ok"}
with open("events.jsonl", "a") as f:
    f.write(json.dumps(event, sort_keys=True) + "\n")
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why one JSON object per line? | What fields are mandatory? | How do you ship events?

Python Lesson: Python Path Handling

Purpose. Use pathlib for clarity and safer path handling.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
from pathlib import Path
root = Path.home() / ".local" / "state" / "greynoc"
root.mkdir(parents=True, exist_ok=True)
report = root / "report.txt"
report.write_text("ok")
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why pathlib? | What permissions are needed? | How do you prevent path traversal?

Python Lesson: Python Unit Tests

Purpose. Tests prevent future edits from breaking operator assumptions.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
# test_validate.py
from validate import is_safe_name
def test_safe_name_accepts_basic():
    assert is_safe_name("server-01")
def test_safe_name_rejects_space():
    assert not is_safe_name("server 01")
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What behavior needs tests? | How do you test failures? | Where do fixtures live?

Python Lesson: Python Packaging Basics

Purpose. Package defensive tools so operators can install and version them consistently.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
# pyproject.toml
[project]
name = "greynoc-defensive-tools"
version = "1.0.0"
dependencies = []
[project.scripts]
greynoc-health = "greynoc_tools.health:main"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.

- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why use pyproject.toml? | What is an entry point? | How do you bump versions?

Python Lesson: Python Virtualenv Deployment

Purpose. Deploy Python scripts with a locked environment and an entrypoint wrapper.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
python3 -m venv /opt/greynoc/venv
/opt/greynoc/venv/bin/python -m pip install --upgrade pip
/opt/greynoc/venv/bin/pip install /opt/greynoc/releases/1.0.0
/opt/greynoc/venv/bin/greynoc-health --help
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why use a venv? | What user owns it? | How do you update safely?

Part 6 - Secure Script Design

Secure design protects the operator, the organization, and the data handled by scripts. This part focuses on abuse resistance, privacy, reliability, idempotence, and observability.

Chapter operator checklist

- Can the reader explain the defensive purpose?
- Can the reader run the lab safely?
- Can the reader verify output?
- Can the reader roll back?
- Can the reader explain what data is handled?

Security Lesson: Command Injection Defense

Purpose. Never concatenate untrusted input into commands. Prefer argument arrays, whitelists, and built-in libraries.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
# Bash: whitelist first
case "$action" in start|stop|status) systemctl "$action" "$service" ;; *) exit 2 ;;
esac
# Python: no shell
subprocess.run(["systemctl", action, service], check=False)
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.

- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Where can input come from? | What is a whitelist? | Why avoid eval?

Security Lesson: Quoting Rules

Purpose. Unquoted variables split on spaces and expand globs. Quote almost every variable expansion.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
for f in "$dir"/*.log; do
  [ -e "$f" ] || continue
  printf '%s\n' "$f"
done
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check When should variables not be quoted? | What happens with spaces? | What is glob expansion?

Security Lesson: Permissions and Umask

Purpose. Set umask for files that may contain operational details. Use explicit chmod for sensitive outputs.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
umask 077
out="case-notes-$(date +%Y%m%d).txt"
touch "$out"
chmod 0600 "$out"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What does umask 077 do? | Why chmod after touch? | Who owns the file?

Security Lesson: Data Minimization

Purpose. Collect the least data needed. More data increases privacy risk, review cost, and breach impact.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.

- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
# Prefer selected logs and time windows:  
journalctl -u ssh -S "2026-06-18 08:00" -U "2026-06-18 10:00" --no-pager
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What data is necessary? | What can be redacted? | How long retain?

Security Lesson: Error Handling

Purpose. A script should fail closed, explain the error, and leave the system in a known state.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
if ! cp -a "$src" "$dst"; then
    echo "copy failed; no cleanup performed" >&2
    exit 1
fi
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is fail closed? | What is safe cleanup? | How should errors be reported?

Security Lesson: Idempotence

Purpose. A script is idempotent if re-running it has the same result without damage. This is critical for automation.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
grep -qxF 'greynoc ALL=(root) NOPASSWD: /usr/bin/systemctl status ssh'
/etc/sudoers.d/greynoc 2>/dev/null ||
echo 'entry missing; manual review required'
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.

- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why is idempotence useful? | What actions are not idempotent? | How do dry runs help?

Security Lesson: Observability

Purpose. Every scheduled script needs logs, metrics, or status output. Silent failures become outages.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
echo "greynoc_health_status{host="$(hostname)"} 1" > /var/tmp/greynoc-health.prom
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Where are metrics scraped? | What does success mean? | How do alerts avoid noise?

Security Lesson: Privacy Review

Purpose. Before sharing a script output, review for usernames, IPs, hostnames, tokens, file paths, and personal data.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
sed -E 's/[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+/x.x.x.x/g' report.txt > report-redacted.txt
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What should be redacted? | What should remain useful? | Who approves sharing?

Security Lesson: Safe Deletion

Purpose. Deletion should require explicit flags, narrow scope, and dry-run. Do not default to destructive behavior.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.

- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
if [[ "${CONFIRM_DELETE:-}" != "yes" ]]; then
    echo "Set CONFIRM_DELETE=yes after dry-run review" >&2
    exit 2
fi
find "$target" -type f -name '*.tmp' -mtime +30 -print -delete
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why require confirmation? | How do you preview? | What paths are forbidden?

Security Lesson: Dependency Trust

Purpose. Dependencies can introduce risk. Prefer standard libraries, pin versions, verify checksums, and monitor updates.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.

- 6 Verify final state with an independent command and document the result.

Reference implementation

```
python -m pip install --require-hashes -r requirements.txt
sha256sum -c checksums.txt
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why pin versions? | What are hashes for? | How do you update safely?

Part 7 - Building, Packaging, Deploying, and Releasing

A script that works on one laptop is not finished. Build and deployment practices turn scripts into reliable operational tools. Use versioning, tests, artifact checksums, staged rollouts, rollback, and release QA.

Chapter operator checklist

- Can the reader explain the defensive purpose?
- Can the reader run the lab safely?
- Can the reader verify output?
- Can the reader roll back?
- Can the reader explain what data is handled?

Build/Deploy Lesson: Repository Layout for Scripting Guides

Purpose. A clean repository layout helps operators find scripts, docs, tests, examples, and release artifacts.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
greynoc-scripts/  
  README.md  
  scripts/  
  lib/  
  tests/  
  examples/  
  docs/  
  packaging/  
  CHANGELOG.md  
  SECURITY.md
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.

- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Where do operator examples live? | What belongs in SECURITY.md? | How do you separate docs from code?

Build/Deploy Lesson: README Template

Purpose. Every script collection should explain purpose, scope, prerequisites, installation, usage, verification, troubleshooting, and support boundaries.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
# GreyNOC Defensive Scripts
Purpose: automate authorized defensive administration.
Scope: owned systems and approved lab environments only.
Install: see docs/install.md.
Usage: run scripts with --help before use.
Verify: run tests and dry-run mode.
Rollback: see docs/rollback.md.
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.

- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What scope language is required? | Where is rollback documented? | How does --help help users?

Build/Deploy Lesson: Semantic Versioning

Purpose. Use MAJOR.MINOR.PATCH. Bump patch for fixes, minor for backward-compatible features, and major for breaking changes.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
1.0.0 initial release
1.0.1 documentation fix
1.1.0 new health-check module
2.0.0 changed config format
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What version bump fits a typo fix? | What bump fits a new command? | What bump fits a config break?

Build/Deploy Lesson: Changelog Discipline

Purpose. A changelog preserves operator trust. It should say what changed, why it matters, and whether action is required.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
## 1.0.1 - 2026-06-18
Fixed
- Corrected service timer example.
- Added missing rollback step for symlink deployment.
Operator action
- No redeploy required unless using the timer example.
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What belongs in Operator action? | Why date releases? | How do you link tickets?

Build/Deploy Lesson: Pre-Commit Checks

Purpose. Run formatting, static analysis, tests, and secret scans before merge.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -euo pipefail
shellcheck scripts/*.sh
bash -n scripts/*.sh
python -m pytest
python -m compileall .
git diff --check
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What does `git diff --check` detect? | Where do secret scans fit? | What must block release?

Build/Deploy Lesson: CI Pipeline

Purpose. A small CI pipeline should lint, test, package, and publish artifacts only after passing all gates.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.

- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
stages:
- lint
- test
- package
- release
release gates:
- tests pass
- changelog updated
- version bumped
- QA checklist signed
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What should never be skipped? | How do artifacts get named? | Who approves release?

Build/Deploy Lesson: Build Artifacts

Purpose. Artifacts should include scripts, docs, checksums, SBOM when applicable, and release notes.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.

- 6 Verify final state with an independent command and document the result.

Reference implementation

```
dist/  
greynoc-defensive-scripts-1.0.0.tar.gz  
greynoc-defensive-scripts-1.0.0.sha256  
RELEASE_NOTES.md  
SBOM.json
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why checksums? | What is an SBOM? | Where should artifacts be stored?

Build/Deploy Lesson: Deployment Rings

Purpose. Deploy in rings: lab, single test host, small pilot, broader fleet. Stop when signals degrade.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
ring0: disposable lab
ring1: one noncritical endpoint
ring2: pilot group
ring3: production fleet
rollback trigger: failed health check, error rate increase, operator report
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why deploy gradually? | What metrics matter? | Who can halt?

Build/Deploy Lesson: Rollback Plan

Purpose. Rollback must be written before deployment. A plan should include command, validation, data impact, and owner.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
ln -sf /opt/greynoc/releases/0.9.3 /opt/greynoc/current
systemctl restart greynoc-health.service
systemctl status greynoc-health.service --no-pager
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.

- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What validates rollback? | How long keep old versions? | When restore data?

Build/Deploy Lesson: Secrets Handling

Purpose. Scripts should read secrets from approved secret stores or protected environment files, never source control.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
# Bad: API_TOKEN="hardcoded"
# Better: read from protected file supplied by deployment system
set -a
. /etc/greynoc/health.env
set +a
: "${API_TOKEN:?missing token}"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.

- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What file permissions are needed? | Why avoid shell history? | How do you rotate secrets?

Build/Deploy Lesson: Release QA Checklist

Purpose. A release candidate should pass repeatable checks: lint, tests, build, install, dry run, rollback, docs review, and artifact verification.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
[ ] shellcheck clean
[ ] unit tests pass
[ ] dry-run verified
[ ] install verified
[ ] logs reviewed
[ ] rollback verified
[ ] docs reviewed
[ ] version and changelog match
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Who signs QA? | What evidence is stored? | What blocks release?

Build/Deploy Lesson: Operator Training Plan

Purpose. Training should mix explanation, guided lab, independent exercise, review, quiz, and remediation. Operators learn fastest when they can break safe lab systems and fix them.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

lesson flow:

1. objective
2. safety note
3. demonstration
4. guided lab
5. independent challenge
6. quiz
7. debrief
8. field checklist

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check Why include debrief? | How do you grade labs? | What counts as mastery?

Part 8 - Operator Micro-Lessons and Drill Cards

The drill cards below are compact lessons that reinforce real operator tasks. Each card can be used as a daily exercise, team quiz, or lab prompt.

Chapter operator checklist

- Can the reader explain the defensive purpose?
- Can the reader run the lab safely?
- Can the reader verify output?
- Can the reader roll back?
- Can the reader explain what data is handled?

Drill Card 01: Operator Aliases And Shell Functions

Purpose. This lesson teaches operator aliases and shell functions as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="operator aliases and shell functions"
log="$HOME/.local/state/greynoc/micro-01.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=01 topic='operator aliases and shell functions'
      status=start" >> "$log"
# Insert the approved command set for: operator aliases and shell functions
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of operator aliases and shell functions? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 02: Environment Variables And Path Hygiene

Purpose. This lesson teaches environment variables and PATH hygiene as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="environment variables and PATH hygiene"
log="$HOME/.local/state/greynoc/micro-02.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=02 topic='environment variables and PATH hygiene'
      status=start" >> "$log"
# Insert the approved command set for: environment variables and PATH hygiene
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of environment variables and PATH hygiene? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 03: Date And Time Handling

Purpose. This lesson teaches date and time handling as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="date and time handling"
log="$HOME/.local/state/greynoc/micro-03.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=03 topic='date and time handling' status=start" >> "$log"
# Insert the approved command set for: date and time handling
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.

- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of date and time handling? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 04: Exit Codes And Runbooks

Purpose. This lesson teaches exit codes and runbooks as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="exit codes and runbooks"
log="$HOME/.local/state/greynoc/micro-04.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=04 topic='exit codes and runbooks' status=start" >> "$log"
# Insert the approved command set for: exit codes and runbooks
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.

- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of exit codes and runbooks? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 05: Regular Expressions For Defenders

Purpose. This lesson teaches regular expressions for defenders as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="regular expressions for defenders"
log="$HOME/.local/state/greynoc/micro-05.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=05 topic='regular expressions for defenders' status=start"
  >> "$log"
# Insert the approved command set for: regular expressions for defenders
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of regular expressions for defenders? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 06: Awk Field Processing

Purpose. This lesson teaches awk field processing as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="awk field processing"
log="$HOME/.local/state/greynoc/micro-06.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=06 topic='awk field processing' status=start" >> "$log"
# Insert the approved command set for: awk field processing
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of awk field processing? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 07: Sed Replacement Safety

Purpose. This lesson teaches sed replacement safety as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="sed replacement safety"
log="$HOME/.local/state/greynoc/micro-07.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=07 topic='sed replacement safety' status=start" >> "$log"
# Insert the approved command set for: sed replacement safety
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of sed replacement safety? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 08: Jq Filters For Events

Purpose. This lesson teaches jq filters for events as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="jq filters for events"
log="$HOME/.local/state/greynoc/micro-08.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=08 topic='jq filters for events' status=start" >> "$log"
# Insert the approved command set for: jq filters for events
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of jq filters for events? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 09: Yq For Yaml Configs

Purpose. This lesson teaches yq for YAML configs as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="yq for YAML configs"
log="$HOME/.local/state/greynoc/micro-09.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=09 topic='yq for YAML configs' status=start" >> "$log"
# Insert the approved command set for: yq for YAML configs
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of yq for YAML configs? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 10: Curl For Internal Apis

Purpose. This lesson teaches curl for internal APIs as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="curl for internal APIs"
log="$HOME/.local/state/greynoc/micro-10.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=10 topic='curl for internal APIs' status=start" >> "$log"
# Insert the approved command set for: curl for internal APIs
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of curl for internal APIs? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 11: Api Retry And Timeout Patterns

Purpose. This lesson teaches API retry and timeout patterns as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="API retry and timeout patterns"
log="$HOME/.local/state/greynoc/micro-11.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=11 topic='API retry and timeout patterns' status=start" >>
"$log"
# Insert the approved command set for: API retry and timeout patterns
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of API retry and timeout patterns? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 12: Rate Limiting

Purpose. This lesson teaches rate limiting as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="rate limiting"
log="$HOME/.local/state/greynoc/micro-12.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=12 topic='rate limiting' status=start" >> "$log"
# Insert the approved command set for: rate limiting
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of rate limiting? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 13: Lock Contention Handling

Purpose. This lesson teaches lock contention handling as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="lock contention handling"
log="$HOME/.local/state/greynoc/micro-13.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=13 topic='lock contention handling' status=start" >> "$log"
# Insert the approved command set for: lock contention handling
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of lock contention handling? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 14: Filesystem Traversal

Purpose. This lesson teaches filesystem traversal as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="filesystem traversal"
log="$HOME/.local/state/greynoc/micro-14.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=14 topic='filesystem traversal' status=start" >> "$log"
# Insert the approved command set for: filesystem traversal
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of filesystem traversal? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 15: Archive Naming Conventions

Purpose. This lesson teaches archive naming conventions as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="archive naming conventions"
log="$HOME/.local/state/greynoc/micro-15.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=15 topic='archive naming conventions' status=start" >> "$log"
# Insert the approved command set for: archive naming conventions
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of archive naming conventions? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 16: Checksum Verification

Purpose. This lesson teaches checksum verification as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="checksum verification"
log="$HOME/.local/state/greynoc/micro-16.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=16 topic='checksum verification' status=start" >> "$log"
# Insert the approved command set for: checksum verification
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of checksum verification? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 17: Encrypted Archive Workflow

Purpose. This lesson teaches encrypted archive workflow as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="encrypted archive workflow"
log="$HOME/.local/state/greynoc/micro-17.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=17 topic='encrypted archive workflow' status=start" >> "$log"
# Insert the approved command set for: encrypted archive workflow
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of encrypted archive workflow? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 18: Gpg Basics For Operators

Purpose. This lesson teaches GPG basics for operators as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="GPG basics for operators"
log="$HOME/.local/state/greynoc/micro-18.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=18 topic='GPG basics for operators' status=start" >> "$log"
# Insert the approved command set for: GPG basics for operators
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of GPG basics for operators? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 19: Ssh Config Management

Purpose. This lesson teaches SSH config management as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="SSH config management"
log="$HOME/.local/state/greynoc/micro-19.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=19 topic='SSH config management' status=start" >> "$log"
# Insert the approved command set for: SSH config management
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of SSH config management? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 20: Known_Hosts Hygiene

Purpose. This lesson teaches known_hosts hygiene as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="known_hosts hygiene"
log="$HOME/.local/state/greynoc/micro-20.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=20 topic='known_hosts hygiene' status=start" >> "$log"
# Insert the approved command set for: known_hosts hygiene
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of known_hosts hygiene? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 21: Authorized_Keys Review

Purpose. This lesson teaches `authorized_keys` review as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="authorized_keys review"
log="$HOME/.local/state/greynoc/micro-21.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=21 topic='authorized_keys review' status=start" >> "$log"
# Insert the approved command set for: authorized_keys review
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of `authorized_keys` review? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 22: Sudo Audit

Purpose. This lesson teaches sudo audit as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="sudo audit"
log="$HOME/.local/state/greynoc/micro-22.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=22 topic='sudo audit' status=start" >> "$log"
# Insert the approved command set for: sudo audit
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of sudo audit? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 23: Service Dependency Mapping

Purpose. This lesson teaches service dependency mapping as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="service dependency mapping"
log="$HOME/.local/state/greynoc/micro-23.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=23 topic='service dependency mapping' status=start" >> "$log"
# Insert the approved command set for: service dependency mapping
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of service dependency mapping? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 24: Port Inventory

Purpose. This lesson teaches port inventory as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="port inventory"
log="$HOME/.local/state/greynoc/micro-24.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=24 topic='port inventory' status=start" >> "$log"
# Insert the approved command set for: port inventory
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of port inventory? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 25: Dns Lookup Scripts

Purpose. This lesson teaches DNS lookup scripts as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="DNS lookup scripts"
log="$HOME/.local/state/greynoc/micro-25.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=25 topic='DNS lookup scripts' status=start" >> "$log"
# Insert the approved command set for: DNS lookup scripts
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of DNS lookup scripts? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 26: Certificate Expiration Checks

Purpose. This lesson teaches certificate expiration checks as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="certificate expiration checks"
log="$HOME/.local/state/greynoc/micro-26.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=26 topic='certificate expiration checks' status=start" >>
"$log"
# Insert the approved command set for: certificate expiration checks
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of certificate expiration checks? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 27: Web Health Checks

Purpose. This lesson teaches web health checks as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="web health checks"
log="$HOME/.local/state/greynoc/micro-27.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=27 topic='web health checks' status=start" >> "$log"
# Insert the approved command set for: web health checks
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of web health checks? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 28: Http Status Dashboards

Purpose. This lesson teaches HTTP status dashboards as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="HTTP status dashboards"
log="$HOME/.local/state/greynoc/micro-28.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=28 topic='HTTP status dashboards' status=start" >> "$log"
# Insert the approved command set for: HTTP status dashboards
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of HTTP status dashboards? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 29: TLS Metadata Review

Purpose. This lesson teaches TLS metadata review as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="TLS metadata review"
log="$HOME/.local/state/greynoc/micro-29.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=29 topic='TLS metadata review' status=start" >> "$log"
# Insert the approved command set for: TLS metadata review
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of TLS metadata review? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 30: Log Parsing Patterns

Purpose. This lesson teaches log parsing patterns as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="log parsing patterns"
log="$HOME/.local/state/greynoc/micro-30.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=30 topic='log parsing patterns' status=start" >> "$log"
# Insert the approved command set for: log parsing patterns
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of log parsing patterns? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 31: Auth Failure Summarization

Purpose. This lesson teaches auth failure summarization as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="auth failure summarization"
log="$HOME/.local/state/greynoc/micro-31.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=31 topic='auth failure summarization' status=start" >> "$log"
# Insert the approved command set for: auth failure summarization
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of auth failure summarization? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 32: Failed Service Summarization

Purpose. This lesson teaches failed service summarization as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="failed service summarization"
log="$HOME/.local/state/greynoc/micro-32.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=32 topic='failed service summarization' status=start" >>
"$log"
# Insert the approved command set for: failed service summarization
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of failed service summarization? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 33: Process Ancestry Review

Purpose. This lesson teaches process ancestry review as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="process ancestry review"
log="$HOME/.local/state/greynoc/micro-33.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=33 topic='process ancestry review' status=start" >> "$log"
# Insert the approved command set for: process ancestry review
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of process ancestry review? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 34: Memory Pressure Snapshot

Purpose. This lesson teaches memory pressure snapshot as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="memory pressure snapshot"
log="$HOME/.local/state/greynoc/micro-34.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=34 topic='memory pressure snapshot' status=start" >> "$log"
# Insert the approved command set for: memory pressure snapshot
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of memory pressure snapshot? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 35: Cpu Pressure Snapshot

Purpose. This lesson teaches CPU pressure snapshot as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="CPU pressure snapshot"
log="$HOME/.local/state/greynoc/micro-35.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=35 topic='CPU pressure snapshot' status=start" >> "$log"
# Insert the approved command set for: CPU pressure snapshot
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of CPU pressure snapshot? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 36: Network Interface Errors

Purpose. This lesson teaches network interface errors as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="network interface errors"
log="$HOME/.local/state/greynoc/micro-36.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=36 topic='network interface errors' status=start" >> "$log"
# Insert the approved command set for: network interface errors
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of network interface errors? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 37: Package Update Reports

Purpose. This lesson teaches package update reports as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="package update reports"
log="$HOME/.local/state/greynoc/micro-37.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=37 topic='package update reports' status=start" >> "$log"
# Insert the approved command set for: package update reports
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of package update reports? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 38: Kernel Version Reports

Purpose. This lesson teaches kernel version reports as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="kernel version reports"
log="$HOME/.local/state/greynoc/micro-38.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=38 topic='kernel version reports' status=start" >> "$log"
# Insert the approved command set for: kernel version reports
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of kernel version reports? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 39: Container Image Inventory

Purpose. This lesson teaches container image inventory as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="container image inventory"
log="$HOME/.local/state/greynoc/micro-39.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=39 topic='container image inventory' status=start" >> "$log"
# Insert the approved command set for: container image inventory
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of container image inventory? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 40: Kubernetes Kubectl Scripting Boundaries

Purpose. This lesson teaches Kubernetes kubectl scripting boundaries as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Kubernetes kubectl scripting boundaries"
log="$HOME/.local/state/greynoc/micro-40.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=40 topic='Kubernetes kubectl scripting boundaries'
      status=start" >> "$log"
# Insert the approved command set for: Kubernetes kubectl scripting boundaries
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Kubernetes kubectl scripting boundaries? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 41: Ansible Wrapper Safety

Purpose. This lesson teaches Ansible wrapper safety as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Ansible wrapper safety"
log="$HOME/.local/state/greynoc/micro-41.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=41 topic='Ansible wrapper safety' status=start" >> "$log"
# Insert the approved command set for: Ansible wrapper safety
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Ansible wrapper safety? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 42: Terraform Plan Review Helper

Purpose. This lesson teaches Terraform plan review helper as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Terraform plan review helper"
log="$HOME/.local/state/greynoc/micro-42.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=42 topic='Terraform plan review helper' status=start" >>
"$log"
# Insert the approved command set for: Terraform plan review helper
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Terraform plan review helper? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 43: Git Branch Release Hygiene

Purpose. This lesson teaches Git branch release hygiene as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Git branch release hygiene"
log="$HOME/.local/state/greynoc/micro-43.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=43 topic='Git branch release hygiene' status=start" >> "$log"
# Insert the approved command set for: Git branch release hygiene
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Git branch release hygiene? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 44: Git Hooks For Docs

Purpose. This lesson teaches Git hooks for docs as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Git hooks for docs"
log="$HOME/.local/state/greynoc/micro-44.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=44 topic='Git hooks for docs' status=start" >> "$log"
# Insert the approved command set for: Git hooks for docs
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Git hooks for docs? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 45: Markdown Lint For Guides

Purpose. This lesson teaches Markdown lint for guides as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Markdown lint for guides"
log="$HOME/.local/state/greynoc/micro-45.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=45 topic='Markdown lint for guides' status=start" >> "$log"
# Insert the approved command set for: Markdown lint for guides
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Markdown lint for guides? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 46: Pdf Release Checklist

Purpose. This lesson teaches PDF release checklist as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="PDF release checklist"
log="$HOME/.local/state/greynoc/micro-46.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=46 topic='PDF release checklist' status=start" >> "$log"
# Insert the approved command set for: PDF release checklist
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of PDF release checklist? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 47: Termux Field Notebook

Purpose. This lesson teaches Termux field notebook as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Termux field notebook"
log="$HOME/.local/state/greynoc/micro-47.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=47 topic='Termux field notebook' status=start" >> "$log"
# Insert the approved command set for: Termux field notebook
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Termux field notebook? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 48: Termux Notification Wrappers

Purpose. This lesson teaches Termux notification wrappers as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Termux notification wrappers"
log="$HOME/.local/state/greynoc/micro-48.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=48 topic='Termux notification wrappers' status=start" >>
"$log"
# Insert the approved command set for: Termux notification wrappers
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Termux notification wrappers? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 49: Termux Backup Retention

Purpose. This lesson teaches Termux backup retention as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Termux backup retention"
log="$HOME/.local/state/greynoc/micro-49.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=49 topic='Termux backup retention' status=start" >> "$log"
# Insert the approved command set for: Termux backup retention
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Termux backup retention? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 50: Termux Ssh Tunnel Warning Boundaries

Purpose. This lesson teaches Termux SSH tunnel warning boundaries as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Termux SSH tunnel warning boundaries"
log="$HOME/.local/state/greynoc/micro-50.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=50 topic='Termux SSH tunnel warning boundaries'
      status=start" >> "$log"
# Insert the approved command set for: Termux SSH tunnel warning boundaries
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Termux SSH tunnel warning boundaries? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 51: Android Storage Permissions

Purpose. This lesson teaches Android storage permissions as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Android storage permissions"
log="$HOME/.local/state/greynoc/micro-51.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=51 topic='Android storage permissions' status=start" >>
"$log"
# Insert the approved command set for: Android storage permissions
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Android storage permissions? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 52: Android Battery Optimization Notes

Purpose. This lesson teaches Android battery optimization notes as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Android battery optimization notes"
log="$HOME/.local/state/greynoc/micro-52.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=52 topic='Android battery optimization notes' status=start"
  >> "$log"
# Insert the approved command set for: Android battery optimization notes
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Android battery optimization notes? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 53: Linux Desktop Operator Scripts

Purpose. This lesson teaches Linux desktop operator scripts as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Linux desktop operator scripts"
log="$HOME/.local/state/greynoc/micro-53.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=53 topic='Linux desktop operator scripts' status=start" >>
"$log"
# Insert the approved command set for: Linux desktop operator scripts
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Linux desktop operator scripts? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 54: Server Operator Scripts

Purpose. This lesson teaches server operator scripts as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="server operator scripts"
log="$HOME/.local/state/greynoc/micro-54.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=54 topic='server operator scripts' status=start" >> "$log"
# Insert the approved command set for: server operator scripts
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of server operator scripts? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 55: Home Lab Scripts

Purpose. This lesson teaches home lab scripts as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="home lab scripts"
log="$HOME/.local/state/greynoc/micro-55.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=55 topic='home lab scripts' status=start" >> "$log"
# Insert the approved command set for: home lab scripts
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of home lab scripts? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 56: Small Business Scripts

Purpose. This lesson teaches small business scripts as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="small business scripts"
log="$HOME/.local/state/greynoc/micro-56.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=56 topic='small business scripts' status=start" >> "$log"
# Insert the approved command set for: small business scripts
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of small business scripts? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 57: Soc Handoff Scripts

Purpose. This lesson teaches SOC handoff scripts as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="SOC handoff scripts"
log="$HOME/.local/state/greynoc/micro-57.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=57 topic='SOC handoff scripts' status=start" >> "$log"
# Insert the approved command set for: SOC handoff scripts
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of SOC handoff scripts? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 58: Noc Status Scripts

Purpose. This lesson teaches NOC status scripts as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="NOC status scripts"
log="$HOME/.local/state/greynoc/micro-58.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=58 topic='NOC status scripts' status=start" >> "$log"
# Insert the approved command set for: NOC status scripts
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of NOC status scripts? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 59: Incident Timeline Scripts

Purpose. This lesson teaches incident timeline scripts as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="incident timeline scripts"
log="$HOME/.local/state/greynoc/micro-59.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=59 topic='incident timeline scripts' status=start" >> "$log"
# Insert the approved command set for: incident timeline scripts
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of incident timeline scripts? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 60: Case Evidence Manifest

Purpose. This lesson teaches case evidence manifest as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="case evidence manifest"
log="$HOME/.local/state/greynoc/micro-60.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=60 topic='case evidence manifest' status=start" >> "$log"
# Insert the approved command set for: case evidence manifest
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of case evidence manifest? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 61: Chain Of Custody Basics

Purpose. This lesson teaches chain of custody basics as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="chain of custody basics"
log="$HOME/.local/state/greynoc/micro-61.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=61 topic='chain of custody basics' status=start" >> "$log"
# Insert the approved command set for: chain of custody basics
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of chain of custody basics? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 62: Defensive Scan Authorization Sheet

Purpose. This lesson teaches defensive scan authorization sheet as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="defensive scan authorization sheet"
log="$HOME/.local/state/greynoc/micro-62.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=62 topic='defensive scan authorization sheet' status=start"
  >> "$log"
# Insert the approved command set for: defensive scan authorization sheet
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of defensive scan authorization sheet? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 63: Script Peer Review

Purpose. This lesson teaches script peer review as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="script peer review"
log="$HOME/.local/state/greynoc/micro-63.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=63 topic='script peer review' status=start" >> "$log"
# Insert the approved command set for: script peer review
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of script peer review? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 64: Script Rollback Tabletop

Purpose. This lesson teaches script rollback tabletop as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="script rollback tabletop"
log="$HOME/.local/state/greynoc/micro-64.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=64 topic='script rollback tabletop' status=start" >> "$log"
# Insert the approved command set for: script rollback tabletop
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of script rollback tabletop? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 65: Operator Quiz Design

Purpose. This lesson teaches operator quiz design as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="operator quiz design"
log="$HOME/.local/state/greynoc/micro-65.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=65 topic='operator quiz design' status=start" >> "$log"
# Insert the approved command set for: operator quiz design
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of operator quiz design? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 66: Lesson Lab Grading

Purpose. This lesson teaches lesson lab grading as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="lesson lab grading"
log="$HOME/.local/state/greynoc/micro-66.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=66 topic='lesson lab grading' status=start" >> "$log"
# Insert the approved command set for: lesson lab grading
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of lesson lab grading? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 67: Maintenance Windows

Purpose. This lesson teaches maintenance windows as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="maintenance windows"
log="$HOME/.local/state/greynoc/micro-67.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=67 topic='maintenance windows' status=start" >> "$log"
# Insert the approved command set for: maintenance windows
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of maintenance windows? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 68: Change Tickets

Purpose. This lesson teaches change tickets as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="change tickets"
log="$HOME/.local/state/greynoc/micro-68.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=68 topic='change tickets' status=start" >> "$log"
# Insert the approved command set for: change tickets
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of change tickets? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 69: Post-Change Validation

Purpose. This lesson teaches post-change validation as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="post-change validation"
log="$HOME/.local/state/greynoc/micro-69.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=69 topic='post-change validation' status=start" >> "$log"
# Insert the approved command set for: post-change validation
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of post-change validation? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 70: Release Notes

Purpose. This lesson teaches release notes as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="release notes"
log="$HOME/.local/state/greynoc/micro-70.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=70 topic='release notes' status=start" >> "$log"
# Insert the approved command set for: release notes
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of release notes? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 71: Version Bump Policy

Purpose. This lesson teaches version bump policy as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="version bump policy"
log="$HOME/.local/state/greynoc/micro-71.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=71 topic='version bump policy' status=start" >> "$log"
# Insert the approved command set for: version bump policy
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of version bump policy? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 72: Artifact Signing

Purpose. This lesson teaches artifact signing as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="artifact signing"
log="$HOME/.local/state/greynoc/micro-72.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=72 topic='artifact signing' status=start" >> "$log"
# Insert the approved command set for: artifact signing
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of artifact signing? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 73: Sbom Note

Purpose. This lesson teaches SBOM note as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="SBOM note"
log="$HOME/.local/state/greynoc/micro-73.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=73 topic='SBOM note' status=start" >> "$log"
# Insert the approved command set for: SBOM note
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of SBOM note? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 74: Containerized Test Harness

Purpose. This lesson teaches containerized test harness as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="containerized test harness"
log="$HOME/.local/state/greynoc/micro-74.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=74 topic='containerized test harness' status=start" >> "$log"
# Insert the approved command set for: containerized test harness
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of containerized test harness? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 75: Shell Unit Tests With Bats

Purpose. This lesson teaches shell unit tests with bats as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="shell unit tests with bats"
log="$HOME/.local/state/greynoc/micro-75.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=75 topic='shell unit tests with bats' status=start" >> "$log"
# Insert the approved command set for: shell unit tests with bats
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of shell unit tests with bats? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 76: Python Pytest Fixtures

Purpose. This lesson teaches Python pytest fixtures as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Python pytest fixtures"
log="$HOME/.local/state/greynoc/micro-76.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=76 topic='Python pytest fixtures' status=start" >> "$log"
# Insert the approved command set for: Python pytest fixtures
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Python pytest fixtures? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 77: Powershell Defensive Comparison

Purpose. This lesson teaches PowerShell defensive comparison as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="PowerShell defensive comparison"
log="$HOME/.local/state/greynoc/micro-77.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=77 topic='PowerShell defensive comparison' status=start" >>
"$log"
# Insert the approved command set for: PowerShell defensive comparison
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of PowerShell defensive comparison? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 78: Windows/Linux Boundary Notes

Purpose. This lesson teaches Windows/Linux boundary notes as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="Windows/Linux boundary notes"
log="$HOME/.local/state/greynoc/micro-78.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=78 topic='Windows/Linux boundary notes' status=start" >>
"$log"
# Insert the approved command set for: Windows/Linux boundary notes
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of Windows/Linux boundary notes? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 79: Cross-Platform Path Notes

Purpose. This lesson teaches cross-platform path notes as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="cross-platform path notes"
log="$HOME/.local/state/greynoc/micro-79.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=79 topic='cross-platform path notes' status=start" >> "$log"
# Insert the approved command set for: cross-platform path notes
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of cross-platform path notes? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 80: Locale And Encoding

Purpose. This lesson teaches locale and encoding as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="locale and encoding"
log="$HOME/.local/state/greynoc/micro-80.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=80 topic='locale and encoding' status=start" >> "$log"
# Insert the approved command set for: locale and encoding
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of locale and encoding? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 81: Timezone Correctness

Purpose. This lesson teaches timezone correctness as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="timezone correctness"
log="$HOME/.local/state/greynoc/micro-81.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=81 topic='timezone correctness' status=start" >> "$log"
# Insert the approved command set for: timezone correctness
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of timezone correctness? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 82: Run As Non-Root

Purpose. This lesson teaches run as non-root as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="run as non-root"
log="$HOME/.local/state/greynoc/micro-82.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=82 topic='run as non-root' status=start" >> "$log"
# Insert the approved command set for: run as non-root
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of run as non-root? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 83: Privilege Separation

Purpose. This lesson teaches privilege separation as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="privilege separation"
log="$HOME/.local/state/greynoc/micro-83.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=83 topic='privilege separation' status=start" >> "$log"
# Insert the approved command set for: privilege separation
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of privilege separation? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 84: Capabilities And Limits

Purpose. This lesson teaches capabilities and limits as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="capabilities and limits"
log="$HOME/.local/state/greynoc/micro-84.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=84 topic='capabilities and limits' status=start" >> "$log"
# Insert the approved command set for: capabilities and limits
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of capabilities and limits? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 85: Read-Only Mounts

Purpose. This lesson teaches read-only mounts as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="read-only mounts"
log="$HOME/.local/state/greynoc/micro-85.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=85 topic='read-only mounts' status=start" >> "$log"
# Insert the approved command set for: read-only mounts
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of read-only mounts? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 86: Configuration Templates

Purpose. This lesson teaches configuration templates as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="configuration templates"
log="$HOME/.local/state/greynoc/micro-86.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=86 topic='configuration templates' status=start" >> "$log"
# Insert the approved command set for: configuration templates
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of configuration templates? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 87: Golden Baselines

Purpose. This lesson teaches golden baselines as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="golden baselines"
log="$HOME/.local/state/greynoc/micro-87.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=87 topic='golden baselines' status=start" >> "$log"
# Insert the approved command set for: golden baselines
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of golden baselines? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 88: Drift Remediation Request

Purpose. This lesson teaches drift remediation request as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="drift remediation request"
log="$HOME/.local/state/greynoc/micro-88.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=88 topic='drift remediation request' status=start" >> "$log"
# Insert the approved command set for: drift remediation request
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of drift remediation request? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 89: Backup Restore Lab

Purpose. This lesson teaches backup restore lab as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="backup restore lab"
log="$HOME/.local/state/greynoc/micro-89.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=89 topic='backup restore lab' status=start" >> "$log"
# Insert the approved command set for: backup restore lab
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of backup restore lab? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 90: Operator Troubleshooting Tree

Purpose. This lesson teaches operator troubleshooting tree as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="operator troubleshooting tree"
log="$HOME/.local/state/greynoc/micro-90.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=90 topic='operator troubleshooting tree' status=start" >>
"$log"
# Insert the approved command set for: operator troubleshooting tree
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of operator troubleshooting tree? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 91: Common Shell Bugs

Purpose. This lesson teaches common shell bugs as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="common shell bugs"
log="$HOME/.local/state/greynoc/micro-91.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=91 topic='common shell bugs' status=start" >> "$log"
# Insert the approved command set for: common shell bugs
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of common shell bugs? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 92: Common Python Bugs

Purpose. This lesson teaches common Python bugs as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="common Python bugs"
log="$HOME/.local/state/greynoc/micro-92.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=92 topic='common Python bugs' status=start" >> "$log"
# Insert the approved command set for: common Python bugs
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of common Python bugs? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 93: Common Termux Bugs

Purpose. This lesson teaches common Termux bugs as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="common Termux bugs"
log="$HOME/.local/state/greynoc/micro-93.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=93 topic='common Termux bugs' status=start" >> "$log"
# Insert the approved command set for: common Termux bugs
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of common Termux bugs? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 94: Common Systemd Bugs

Purpose. This lesson teaches common systemd bugs as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="common systemd bugs"
log="$HOME/.local/state/greynoc/micro-94.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=94 topic='common systemd bugs' status=start" >> "$log"
# Insert the approved command set for: common systemd bugs
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of common systemd bugs? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 95: Emergency Stop Switch

Purpose. This lesson teaches emergency stop switch as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="emergency stop switch"
log="$HOME/.local/state/greynoc/micro-95.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=95 topic='emergency stop switch' status=start" >> "$log"
# Insert the approved command set for: emergency stop switch
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of emergency stop switch? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 96: Alert Fatigue Control

Purpose. This lesson teaches alert fatigue control as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="alert fatigue control"
log="$HOME/.local/state/greynoc/micro-96.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=96 topic='alert fatigue control' status=start" >> "$log"
# Insert the approved command set for: alert fatigue control
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of alert fatigue control? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 97: Documentation Debt

Purpose. This lesson teaches documentation debt as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="documentation debt"
log="$HOME/.local/state/greynoc/micro-97.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=97 topic='documentation debt' status=start" >> "$log"
# Insert the approved command set for: documentation debt
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of documentation debt? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 98: On-Call Handoff

Purpose. This lesson teaches on-call handoff as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="on-call handoff"
log="$HOME/.local/state/greynoc/micro-98.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=98 topic='on-call handoff' status=start" >> "$log"
# Insert the approved command set for: on-call handoff
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of on-call handoff? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Drill Card 99: Continuous Improvement

Purpose. This lesson teaches continuous improvement as a defensive operator capability. The goal is to make the task repeatable, reviewable, and safe enough for lab use before production.

Operator framing. Confirm authorization, asset owner, data sensitivity, and rollback before using the pattern. Start read-only, capture evidence, and change only the narrowest required scope.

Core procedure

- 1 Write down the expected result in one sentence.
- 2 Run the smallest read-only command that proves the starting state.
- 3 Add validation for every external input: paths, hostnames, service names, files, API data, and command flags.
- 4 Add logging that records time, host, action, result, and error without exposing secrets.
- 5 Run in dry-run mode, review output, then execute with approval when applicable.
- 6 Verify final state with an independent command and document the result.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
section="continuous improvement"
log="$HOME/.local/state/greynoc/micro-99.log"
mkdir -p "$(dirname "$log")"
echo "$(date -Is) lesson=99 topic='continuous improvement' status=start" >> "$log"
# Insert the approved command set for: continuous improvement
# Always run with --dry-run or read-only mode first.
echo "review scope, inputs, permissions, expected output, rollback" >> "$log"
```

Verification checklist

- Script exits non-zero when a required input is missing or invalid.
- Script output is understandable by another operator.
- Logs are protected and do not contain credentials, tokens, private keys, or unnecessary personal data.
- The command can be rerun safely or clearly refuses to rerun when unsafe.
- Rollback or stop instructions are documented next to the script.

Common failure modes

- Assuming interactive shell variables exist in cron, systemd, SSH, or Termux boot sessions.
- Using unquoted variables, predictable temporary files, or broad globs.
- Collecting too much data and creating privacy or retention problems.
- Treating a successful command exit as proof that the business outcome succeeded.
- Skipping peer review because the script is short.

Knowledge check What is the defensive purpose of continuous improvement? | What input validation is required? | What evidence should be logged? | What rollback or stop condition applies?

Part 9 - Team-Based Operations Scripting (Red, Blue, Purple, White)

Team-based engagements split defensive and offensive-simulation work across four roles. Red operators emulate an authorized adversary; Blue operators detect, triage, and respond; Purple operators run the feedback loop that turns red findings into blue detections; and White operators control the exercise, keep it safe, and score it. This part covers the scripting that makes those roles disciplined and auditable. Consistent with the rest of this manual, every pattern here assumes a signed scope, written authorization, and an owned or explicitly permitted range. Nothing in this part provides exploits, payloads, command-and-control, credential theft, or stealth; the red-team material is engagement scaffolding and guardrails, not offensive tooling.

Chapter operator checklist

- Can the reader explain the defensive purpose?
- Can the reader run the lab safely?
- Can the reader verify output?
- Can the reader roll back?
- Can the reader explain what data is handled?

Red Team - Authorized Adversary Emulation

Red operators emulate a defined threat to test defenses. In a responsible program the value is not in the exploit; it is in disciplined scope control, deconfliction, and evidence so that Blue and White can learn from the activity. The scripts below enforce those disciplines. They do not generate, deliver, or conceal attacks.

Red Lesson: Scope Guard and Target Allow-List

Purpose. Wrap engagement tooling so it physically refuses to act on any host, range, or domain that is not on the signed, hash-pinned scope allow-list. The guard fails closed: unknown target means stop.

Operator framing. Confirm the allow-list is the one signed in the rules of engagement, verify its checksum, and run the guard read-only before any active step.

Core procedure

- 1 Load the approved scope file and verify its SHA-256 against the value recorded in the ROE.
- 2 Normalize each requested target (resolve, strip ports, lower-case) before comparison.
- 3 Match against the allow-list using exact host and CIDR membership, never substring matching.
- 4 Refuse and log any target that is not an explicit match; exit non-zero.
- 5 Record every decision (target, verdict, timestamp, operator) to the engagement log.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
scope="${SCOPE_FILE:?signed scope file required}"
want_sha="${SCOPE_SHA256:?expected scope hash required}"
got_sha="$(sha256sum "$scope" | awk '{print $1}')"
[ "$got_sha" = "$want_sha" ] || { echo "scope hash mismatch - refusing" >&2; exit 3; }
target="${1:?target required}"
if grep -Fxq -- "$target" "$scope"; then
    echo "$(date -Is) target=$target verdict=in-scope op=$USER" >> engagement.log
else
    echo "$(date -Is) target=$target verdict=OUT-OF-SCOPE op=$USER" >> engagement.log
    echo "refusing: $target is not in signed scope" >&2; exit 4
fi
```

Verification checklist

- Guard exits non-zero on a target that is absent from the allow-list.
- Guard exits non-zero when the scope file hash does not match the ROE value.
- Every decision, allow or deny, is written to the engagement log with operator and time.
- No active command runs before the guard returns success.

Common failure modes

- Substring or regex matching that lets a look-alike host slip through.
- Trusting a scope file that was edited after the ROE was signed.
- Logging only denials, so allowed actions have no audit trail.

Knowledge check Why must the guard fail closed? | How is the scope file integrity verified? | Why is CIDR/exact matching safer than substring matching?

Red Lesson: Rules-of-Engagement and Deconfliction Log

Purpose. Maintain a single append-only record of authorization windows, allowed techniques, contacts, and a deconfliction signal so Blue and White can always tell exercise activity from a real incident.

Operator framing. Confirm the ROE window is current, the White cell contact is reachable, and the deconfliction channel is agreed before any activity.

Core procedure

- 1 Read the ROE record: window start/end, allowed activity classes, and emergency contacts.
- 2 Refuse to proceed outside the authorized time window.
- 3 Emit a deconfliction marker (ticket id, source host, activity class) to the agreed channel.
- 4 Append every operator action to a tamper-evident, timestamped log.
- 5 On stop or abort, write a closing entry and notify the White cell.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
now=$(date +%s)
start=$(date -d "$ROE_START" +%s); end=$(date -d "$ROE_END" +%s)
(( now >= start && now <= end )) || { echo "outside ROE window - stop" >&2; exit 5; }
marker="ticket=$ROE_TICKET src=$(hostname) class=${1:?activity class} op=$USER"
echo "$(date -Is) DECONFLICT $marker" | tee -a roe.log >> "$DECONFLICT_CHANNEL"
echo "$(date -Is) ACTION $marker detail='${2:-n/a}'" >> roe.log
```

Verification checklist

- Activity is refused outside the authorized time window.
- Each action produces a deconfliction marker on the agreed channel.
- The ROE log is append-only and includes ticket, source, class, and operator.
- An abort writes a closing record and signals the White cell.

Common failure modes

- Starting activity before the window opens or after it closes.
- Silent activity that Blue could mistake for a genuine intrusion.
- A log that can be rewritten, destroying the audit trail.

Knowledge check What distinguishes exercise traffic from a real incident? | Why is an append-only log required? | Who must be notified on abort?

Red Lesson: Safe Engagement Evidence Capture

Purpose. Capture proof of each authorized step (command, timestamp, result hash) without collecting third-party personal data, and store it encrypted for the report.

Operator framing. Confirm the evidence store is access-controlled, capture only what the report needs, and redact incidental personal data on the way in.

Core procedure

- 1 Record the exact command run, the operator, and an ISO-8601 timestamp.
- 2 Hash captured artifacts (SHA-256) so tampering is detectable later.
- 3 Redact obvious personal data fields before writing to the store.
- 4 Write to an access-controlled directory with restrictive permissions.
- 5 Produce a manifest the report writer can verify independently.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
umask 077
store="${EVIDENCE_DIR:?evidence dir required}"; mkdir -p "$store"
cmd="${1:?command required}"
out="$store/$(date +%s).out"
eval "$cmd" > "$out" 2>&1 || true # capture result, never hide failure
h=$(sha256sum "$out" | awk '{print $1}')
echo "$(date -Is) op=$USER cmd='$cmd' sha256=$h" >> "$store/manifest.log"
```

Verification checklist

- Artifacts are hashed and listed in a verifiable manifest.
- The store is created with restrictive (077) permissions.
- Captured output excludes unnecessary third-party personal data.
- Command failures are still captured, never silently dropped.

Common failure modes

- Sweeping whole directories and capturing unrelated personal data.
- Storing evidence world-readable or in a shared temp path.
- Capturing only successes, which biases the report.

Knowledge check Why hash each artifact? | What data should be redacted at capture time? | Why keep failed-step output too?

Blue Team - Detection, Triage, and Response

Blue operators turn raw telemetry into reliable signal and a repeatable response. The scripts below normalize logs, baseline a system, and score alerts so an on-call operator spends time on what matters. All of it is read-first and works on owned or authorized systems.

Blue Lesson: Log Normalization Pipeline

Purpose. Convert messy multi-source logs into one consistent, queryable schema so detections and triage do not depend on each source's quirks.

Operator framing. Confirm you have read access to the source logs, that retention rules allow processing, and that output excludes secrets.

Core procedure

- 1 Define a small target schema: time, host, source, severity, actor, action, detail.
- 2 Parse each source into the schema; drop or hash fields that carry secrets.
- 3 Emit newline-delimited JSON so downstream tools can stream it.
- 4 Validate that every record has a parseable timestamp and known source.
- 5 Send malformed lines to a quarantine file for review, not into the clean stream.

Reference implementation

```
#!/usr/bin/env python3
import json, sys, datetime
SCHEMA = ('ts', 'host', 'source', 'severity', 'actor', 'action', 'detail')
def normalize(line, source):
    parts = line.rstrip().split(' ', 5)
    if len(parts) < 2: raise ValueError('too short')
    ts = datetime.datetime.fromisoformat(parts[0]).isoformat()
    rec = dict(zip(SCHEMA, [ts, parts[1], source] + parts[2:]))
    return {k: rec.get(k, '') for k in SCHEMA}
for line in sys.stdin:
    try:
        print(json.dumps(normalize(line, sys.argv[1])))
    except Exception as e:
        sys.stderr.write(f'quarantine: {e}: {line}')
```

Verification checklist

- Every clean record validates against the target schema.
- Malformed lines go to quarantine, never silently into the stream.
- Secret-bearing fields are dropped or hashed before output.
- Output is streamable newline-delimited JSON.

Common failure modes

- Assuming every source uses the same time format.
- Letting unparseable lines vanish, hiding ingestion gaps.
- Copying raw tokens or keys straight into the normalized record.

Knowledge check Why normalize to one schema? | Why quarantine instead of drop? | What belongs in the detail field and what must never?

Blue Lesson: File Integrity Baseline and Diff

Purpose. Record a trusted baseline of critical files and compare current state against it so unexpected change is visible, with no agent required.

Operator framing. Confirm the baseline was taken from a known-good system and is stored where an attacker on the monitored host cannot quietly rewrite it.

Core procedure

- 1 Enumerate the critical path list from a reviewed config, not a wildcard sweep.
- 2 Record path, size, mode, owner, and SHA-256 for each file in the baseline.
- 3 Store the baseline off-host or read-only so it stays trustworthy.
- 4 On each run, recompute and diff against the baseline.
- 5 Report added, removed, and changed files with old/new hashes.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
mode="${1:?baseline|check}"; list="${CRIT_LIST:?critical path list}"
snap(){ while read -r f; do [ -e "$f" ] && \
    printf '%s %s\n' "${sha256sum "$f" | awk '{print $1}')" "$f"; done < "$list"; }
case "$mode" in
    baseline) snap > baseline.sha; echo 'baseline written' ;;
    check) snap > current.sha; diff baseline.sha current.sha && echo 'no change' \
        || echo 'integrity drift detected (review diff above)' ;;
esac
```

Verification checklist

- The baseline is stored where the monitored host cannot rewrite it.
- Checks report added, removed, and changed files distinctly.
- The path list is reviewed, not a broad recursive wildcard.
- A drift result is clearly flagged and non-silent.

Common failure modes

- Keeping the baseline on the same host an attacker could edit.
- Hashing so many files that real drift is lost in noise.
- Treating a missing file as 'no change'.

Knowledge check Why store the baseline off-host? | Why a reviewed path list over a wildcard? | How do you distinguish added from changed files?

Blue Lesson: Alert Triage Scorer

Purpose. Rank incoming alerts by a small, explainable score so on-call attention goes to the highest signal first and alert fatigue drops.

Operator framing. Confirm the scoring weights are agreed with the team and reviewed, and that the score is explainable, not a black box.

Core procedure

- 1 Define weighted factors: asset criticality, signal confidence, blast radius, novelty.
- 2 Read each normalized alert and compute a bounded score from the factors.
- 3 Attach the factor breakdown to the alert so the triager sees the why.
- 4 Sort the queue by score and surface the top items.
- 5 Log scoring decisions so weights can be tuned from real outcomes.

Reference implementation

```
#!/usr/bin/env python3
import json, sys
W = {'asset': 3, 'confidence': 2, 'blast': 2, 'novelty': 1}
def score(a):
    s = sum(W[k] * a.get(k, 0) for k in W)
    return min(s, 10), {k: a.get(k, 0) for k in W}
alerts = [json.loads(l) for l in sys.stdin if l.strip()]
for a in alerts:
    a['score'], a['why'] = score(a)
for a in sorted(alerts, key=lambda x: x['score'], reverse=True):
    print(json.dumps(a))
```

Verification checklist

- The score is bounded and reproducible from the inputs.
- Each alert carries its factor breakdown for the triager.
- Scoring weights are reviewed and version-controlled.
- Decisions are logged so weights can be tuned over time.

Common failure modes

- Hidden weights nobody can explain during an incident.
- Unbounded scores that make ranking meaningless.
- Tuning weights silently with no record of why.

Knowledge check Why must the score be explainable? | What factors drive triage priority? | How do logged decisions improve the scorer?

Purple Team - Detection Feedback Loop

Purple operators close the gap between what Red did and what Blue saw. The scripts below map authorized test activity to technique IDs, score detection coverage, and track the gaps to closure. They consume recorded exercise telemetry; they do not run attacks.

Purple Lesson: Technique-to-Detection Coverage Matrix

Purpose. Build a matrix that pairs each technique exercised in an authorized test with whether a detection fired, so coverage and gaps are visible at a glance.

Operator framing. Confirm the exercise log and the detection log both come from the authorized run and share a common time and ticket reference.

Core procedure

- 1 List the techniques the engagement was authorized to exercise (by ID).
- 2 Read the recorded detections and tag each with the technique it covers.
- 3 Join exercised techniques to detections on technique ID and time window.
- 4 Mark each technique covered, partial, or gap.
- 5 Output a sorted matrix and a single coverage percentage.

Reference implementation


```
#!/usr/bin/env python3
import json, sys
exercised = json.load(open(sys.argv[1])) # ['T1059', 'T1053', ...]
detections = json.load(open(sys.argv[2])) # [{'technique': 'T1059'}, ...]
fired = {d['technique'] for d in detections}
rows = [(t, 'covered' if t in fired else 'GAP') for t in sorted(exercised)]
for t, status in rows:
    print(f'{t:10} {status}')
cov = sum(1 for _, s in rows if s == 'covered') / max(len(rows), 1)
print(f'coverage={cov:.0%}')
```

Verification checklist

- Every authorized technique appears in the matrix exactly once.
- Coverage status is one of covered, partial, or gap.
- The join uses technique ID and time window, not name text.
- A single coverage percentage is produced for the report.

Common failure modes

- Counting a detection that fired outside the exercise window.
- Matching on free-text names instead of stable technique IDs.
- Reporting coverage without listing the actual gaps.

Knowledge check Why join on technique ID and time window? | What does a gap row tell Blue? | Why report the explicit gap list, not just a percentage?

Purple Lesson: Detection Test Harness (Replay)

Purpose. Replay a recorded, benign telemetry sample from an authorized exercise against a detection rule to confirm it still fires after changes.

Operator framing. Confirm the sample is benign recorded telemetry, not live activity, and that replay targets a test pipeline, never production response.

Core procedure

- 1 Store a small, benign recorded sample per technique under version control.
- 2 Point the harness at a test detection pipeline, isolated from real response.
- 3 Replay each sample and capture whether the expected rule fired.
- 4 Compare results to the expected outcome and flag regressions.
- 5 Record pass/fail per rule so detection changes are gated by it.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
pipeline="${TEST_PIPELINE:?test pipeline endpoint}"
pass=0; fail=0
for sample in samples/*.ndjson; do
    rule=$(basename "$sample" .ndjson)
    if cat "$sample" | feed-to "$pipeline" | grep -q "rule=$rule"; then
        echo "PASS $rule"; pass=$((pass+1))
    else
        echo "FAIL $rule"; fail=$((fail+1))
    fi
done
echo "pass=$pass fail=$fail"; [ "$fail" -eq 0 ]
```

Verification checklist

- Replay targets a test pipeline isolated from real response.
- Samples are benign, recorded, and version-controlled.
- Each rule has an expected outcome the harness checks.
- A regression (fail) makes the run exit non-zero.

Common failure modes

- Replaying into the live response pipeline and paging on-call.
- Using samples that contain live or sensitive data.
- Letting a silently broken rule pass because nothing checked it.

Knowledge check Why must replay hit a test pipeline only? | What makes a sample safe to store? | How does the harness gate detection changes?

Purple Lesson: Gap Tracker to Closure

Purpose. Turn coverage gaps into tracked work items with an owner, a fix, and a retest date so the loop actually closes instead of being rediscovered next exercise.

Operator framing. Confirm each gap has a named owner and that the tracker is the single source of truth shared by Red, Blue, and White.

Core procedure

- 1 Create one item per gap with technique, description, owner, and due date.
- 2 Link each item to the proposed detection or hardening change.
- 3 Track status: open, in-progress, retest-pending, closed.
- 4 Require a passing replay test before an item can move to closed.
- 5 Report aging open items so nothing silently lingers.

Reference implementation

```
#!/usr/bin/env python3
import json, sys, datetime
items = json.load(open('gaps.json'))
today = datetime.date.today().isoformat()
for g in items:
    if g['status'] == 'closed' and not g.get('retest_passed'):
        sys.exit(f"refuse close {g['technique']}: no passing retest")
    overdue = g['status'] != 'closed' and g['due'] < today
    flag = ' OVERDUE' if overdue else ''
    print(f"{g['technique']:10} {g['status']:14} owner={g['owner']}{flag}")
```

Verification checklist

- Every gap has an owner and a due date.
- Closing an item requires a passing retest.
- Overdue open items are flagged on each report.
- The tracker is the shared single source of truth.

Common failure modes

- Closing gaps without proving the detection now fires.
- Ownerless items that no one drives to closure.
- Letting items age silently past their due date.

Knowledge check Why gate closure on a passing retest? | Why does each gap need a named owner? | What stops the loop from quietly reopening every exercise?

White Team - Exercise Control and Governance

White operators run the exercise: they schedule injects, keep everyone safe, hold the deconfliction registry, score the event, and can stop it instantly. The scripts below make that control reliable and auditable. Safety and the abort path come first.

White Lesson: Safety and Abort Broadcast

Purpose. Provide a single, fast, logged way to halt all exercise activity and tell every team, because a clean abort is the most important control in the room.

Operator framing. Confirm every team is subscribed to the abort channel before the exercise starts and that the abort path is tested in the dry run.

Core procedure

- 1 Define one authoritative abort channel every team subscribes to.
- 2 On abort, broadcast a clear STOP with reason, time, and caller.
- 3 Write the abort to the immutable exercise log first, then notify.
- 4 Require acknowledgement from each team and track who has stopped.
- 5 Do not allow the exercise to resume without an explicit, logged restart.

Reference implementation

```
#!/usr/bin/env bash
set -Eeuo pipefail
reason="${1:?abort reason required}"
msg="STOP EXERCISE reason='$reason' by=$USER at=$(date -Is)"
echo "$msg" >> exercise.log          # log first
for team in red blue purple white; do
    printf '%s\n' "$msg" >> "channels/$team.abort"
done
echo 'abort broadcast sent; awaiting acknowledgements'
```

Verification checklist

- The abort writes to the immutable log before notifying.
- Every team receives the STOP on the shared channel.
- Acknowledgements are tracked until all teams confirm stop.
- Resuming requires an explicit, logged restart.

Common failure modes

- An abort path that was never tested in the dry run.
- Notifying teams but forgetting to record the abort.
- Letting activity drift back on without a logged restart.

Knowledge check Why log the abort before broadcasting? | Why track per-team acknowledgement? | Why must restart be explicit and logged?

White Lesson: Inject Scheduler

Purpose. Release scenario injects on a controlled timeline so the exercise unfolds as designed and every inject is recorded for the after-action review.

Operator framing. Confirm the inject plan is approved, each inject is safe and in scope, and the schedule can be paused by the abort control.

Core procedure

- 1 Load the approved inject plan: id, time offset, target team, content.
- 2 Refuse to fire if a global pause or abort flag is set.
- 3 Release each inject at its scheduled offset from exercise start.
- 4 Log the actual release time and recipient for every inject.
- 5 Skip and flag any inject whose preconditions are not met.

Reference implementation

```
#!/usr/bin/env python3
import json, time, os, datetime
plan = json.load(open('injects.json')) # [{'id', 'offset', 'team', 'text'}]
start = time.time()
for inj in sorted(plan, key=lambda x: x['offset']):
    while time.time() - start < inj['offset']:
        if os.path.exists('ABORT'): raise SystemExit('aborted')
        time.sleep(1)
    line = f"{datetime.datetime.now().isoformat()} inject={inj['id']}
           team={inj['team']}"
    open(f"channels/{inj['team']}.inbox", 'a').write(inj['text'] + '\n')
    open('exercise.log', 'a').write(line + '\n')
```

Verification checklist

- The scheduler halts immediately when the abort flag is set.
- Each inject's actual release time and recipient are logged.
- Injects fire in approved order on their offsets.
- Unmet-precondition injects are skipped and flagged, not forced.

Common failure modes

- A scheduler that ignores the abort flag mid-run.
- Firing injects with no record of when they actually went out.
- Hard-coding the plan instead of loading the approved file.

Knowledge check Why must the scheduler honor the abort flag? | Why log actual vs planned release time? | What should happen when an inject's preconditions are not met?

White Lesson: Scoring and After-Action Tally

Purpose. Score the exercise against agreed objectives and produce an after-action summary, so the event ends with measurable findings rather than opinions.

Operator framing. Confirm the scoring rubric was agreed before the exercise and that scores trace back to logged events, not memory.

Core procedure

- 1 Define objectives and point values agreed by all teams beforehand.
- 2 Derive each score from logged, timestamped exercise events.
- 3 Tally per-objective and overall scores deterministically.
- 4 Attach the supporting log references to each scored objective.
- 5 Emit an after-action summary with scores, gaps, and next steps.

Reference implementation

```
#!/usr/bin/env python3
import json
rubric = json.load(open('rubric.json'))      # {'objective': points}
events = [l.split() for l in open('exercise.log')]
hit = {o for parts in events for o in rubric if any(o in p for p in parts)}
score = sum(pts for o, pts in rubric.items() if o in hit)
total = sum(rubric.values())
for o, pts in rubric.items():
    print(f"{o:24} {'MET' if o in hit else 'missed':6} {pts}")
print(f"score={score}/{total}")
```

Verification checklist

- Every score traces to a logged, timestamped event.
- The rubric was fixed before the exercise began.
- Per-objective and overall tallies are deterministic.
- The after-action summary lists gaps and next steps.

Common failure modes

- Scoring from memory or impressions after the fact.
- Changing the rubric once results are known.
- Reporting a number with no link to the evidence behind it.

Knowledge check Why fix the rubric beforehand? | Why must scores trace to the log? | What turns a score into a useful after-action finding?

Capstone Labs and Final Knowledge Checks

Lab	Objective	Deliverable
Capstone 1	Build a Linux health-check bundle with strict mode, logging, dry-run, and service checks.	Script, README, sample log, rollback note.
Capstone 2	Build a Termux visible keepalive and field-note workflow for an owned Android device.	Scripts, stop command, battery-behavior notes, proof of notification.
Capstone 3	Package and deploy a versioned release into a Linux lab VM using a symlinked release folder with rollback.	Release folder, checksum, changelog, QA checklist.
Capstone 4	Create a log-collection bundle for a simulated incident without collecting unnecessary private data.	Case manifest, redaction note, archive hash.
Capstone 5 (Blue)	Build a detection-and-triage bundle for an owned lab: log normalizer, integrity baseline diff, and alert scorer.	Detection rules, sample alerts, triage log, tuning notes.
Capstone 6 (Purple)	Produce an ATT&CK-style detection-coverage report from a recorded authorized exercise.	Technique-to-detection matrix, coverage score, gap list, retest plan.
Capstone 7 (Red/White)	Build an engagement scope-guard and exercise-control kit for an authorized lab range.	Signed scope allow-list, ROE/deconfliction log, abort broadcast, after-action template.

Final written quiz

Thirty distinct questions spanning the manual. Answer in your own words with a command or short pseudocode example where relevant, then check against the lessons.

1. State the four things you confirm before running any script (owner, asset, objective, rollback). Why each?
2. Explain what `set -Eeuo pipefail` changes and give one case where a local exception is justified.
3. Why is `IFS=$'\n\t'` used in strict-mode Bash, and what failure does it prevent?
4. Write a `getopts` loop that rejects unknown flags and exits non-zero with usage. Why that exit code?
5. Show how to validate a path argument before acting on it, and explain why substring checks are unsafe.
6. Give a logging line that records time, host, action, and result without exposing secrets.
7. Explain dry-run mode and why an operator runs it before changing state.
8. How do you verify final state with an independent command rather than trusting exit code 0?
9. Describe a Termux keepalive that is visible and easy to stop. Why must it never hide activity?
10. How would you back up only user-approved folders in Termux instead of sweeping all storage?
11. Write a systemd unit or cron entry and explain how its environment differs from an interactive shell.
12. Explain idempotence and give a command pattern that is safe to rerun.
13. Describe safe deletion: how do you avoid destructive globs and confirm scope first?
14. How do you minimize collected data to reduce privacy and retention risk?

15. Explain command-injection defense in shell and one concrete quoting rule that prevents it.
16. What does umask 077 do for an evidence or log directory, and why does it matter?
17. Describe semantic versioning and justify the version bump made for this release.
18. What belongs in a changelog entry, and why is changelog discipline a release control?
19. Outline a deployment-ring strategy and where rollback fits.
20. How are secrets handled across build and deploy without landing in source or logs?
21. Red team: why must a scope guard fail closed, and how is the scope file's integrity verified?
22. Red team: what distinguishes authorized exercise activity from a real incident on the wire?
23. Blue team: why normalize multi-source logs to one schema, and what must never enter the detail field?
24. Blue team: why store a file-integrity baseline off-host, and how do you tell added from changed files?
25. Blue team: why must an alert triage score be bounded and explainable during an incident?
26. Purple team: why join exercised techniques to detections on technique ID and time window, not names?
27. Purple team: why must a detection replay harness target a test pipeline rather than live response?
28. Purple team: why gate gap closure on a passing retest, and why does each gap need a named owner?
29. White team: why is the abort broadcast logged before it is sent, and why track per-team acknowledgement?
30. White team: why fix the scoring rubric before the exercise and require scores to trace to the log?

References, QA/QC Record, and Release Notes

- GreyNOC Operator Guides README: repository purpose, active guide list, responsible-use scope, and suggested guide format.
- ShellCheck project: shell static analysis that finds bugs in shell scripts; available through common package managers.
- Termux project documentation and Android platform behavior should be checked on the operator device before production use, because Android version, app source, and battery policy affect behavior.
- Linux distribution manuals for systemd, cron, package managers, auditd, journalctl, SSH, and sudoers should be treated as platform-specific authorities.
- MITRE ATT&CK is referenced in Part 9 only as a technique-naming convention for coverage mapping in authorized exercises.

Release QA/QC Log

Stage	Result	Fix / release action
v1.0.0 release QA	Public release published.	Approved for release.
v1.1.0 content review	Final quiz repeated one question 60 times; capstone table text overlapped; strict-mode IFS tab rendered as a box; no document-control/version-history front matter; no team-based scripting content.	Replaced quiz with 30 distinct questions; rebuilt capstone table with wrapped cells; corrected IFS to <code>\$'\n\t'</code> ; added Document Control and Version History; authored Part 9.
v1.1.0 release QA	Version stamp consistent on every page; new content renders; branding and safety boundary intact; no draft markers.	Approved for release.

Release version: v1.1.0. This manual is a defensive training artifact. Operators must validate commands in a controlled lab, then adapt them to local policy, approved tools, and platform-specific documentation. All Part 9 team activity requires written authorization, an agreed scope, and deconfliction.